COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

PACKAGE 07 / VOLUME 06

Game Systems and Production

Volume 06 • Expanded student handbook

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Volume 06 at a glance 5

Evidence tiers . . . 5

Module 26: Collision detection and physical simulation 6

Lesson 26.1: Broad phase, narrow phase and geometric robustness . . . 7

Lesson 26.2: Rigid bodies, angular motion and collision impulses . . . 10

Lesson 26.3: Constraints, iterative solvers and continuous collision detection . . . 13

Lesson 26.4: Physics testing, determinism and engine integration . . . 16

Module 26 project . . . 18

Module 26 review . . . 19

Module 27: Animation, audio and interactive timing 20

Lesson 27.1: Skeletons, transforms and skinning . . . 21

Lesson 27.2: Animation blending, state machines and inverse kinematics . . . 24

Lesson 27.3: Digital audio, mixing and real-time callback discipline . . . 27

Lesson 27.4: Spatial audio, animation events and multimodal accessibility . . . 30

Module 27 project . . . 32

Module 27 review . . . 33

Module 28: Game AI, decision systems and machine learning 34

Lesson 28.1: Navigation graphs, steering and dynamic obstacles . . . 35

Lesson 28.2: Behavior trees, utility systems and explainable decisions . . . 38

Lesson 28.3: Adversarial search, planning and simulation budgets . . . 41

Lesson 28.4: Learning systems, neural inference and responsible evaluation . . . 44

Module 28 project . . . 46

Module 28 review . . . 47

Module 29: Multiplayer simulation and online services 48

Lesson 29.1: Authority, prediction and reconciliation . . . 49

Lesson 29.2: Snapshots, interpolation, relevance and lag compensation . . . 52

Lesson 29.3: Lockstep, rollback and deterministic replay . . . 55

Lesson 29.4: Match services, identity, persistence and failure recovery . . . 58

Module 29 project . . . 60

Module 29 review . . . 61

Module 30: Production quality and the undergraduate engine capstone 62

Lesson 30.1: Profiling, frame budgets and performance regression tests . . . 63

Lesson 30.2: Compatibility, accessibility, localization and human testing . . . 66

Lesson 30.3: Packaging, patching, modding and supply-chain boundaries . . . 69

Lesson 30.4: Capstone review, release and technical postmortem . . . 72

Module 30 project . . . 74

Module 30 review . . . 75

Reference Directory 76

S01: Collision documentation . . . 76

S02: Simulation documentation . . . 76

S03: glTF tutorial: Skins . . . 76

S04: glTF 2.0 specification . . . 76

S05: Writing a callback function 76

S06: SDL_AudioStreamCallback 76

S07: Xbox Accessibility Guidelines 76

S08: CS188: Informed search 77

S09: Introduction 77

S10: CS188: Minimax 77

S11: Automatic differentiation with torch.autograd 77

S12: Networked movement in the Character Movement Component 77

S13: Snapshot Interpolation 77

S14: GGPO SDK repository 77

S15: Transaction documentation 78

S16: Atomic Commit in SQLite 78

S17: os.replace documentation 78

S18: zipfile documentation 78

S19: Fix Your Timestep! 78

S23: XAG 103: Additional channels for audio and visual cues 78

S24: Deterministic Lockstep 78

S25: XAG 107: Input 78

S26: XAG 101: Text display 79

Original program reading IDs 79

Volume 06 at a glance

Game Systems and Production connects simulation, animation, audio, AI and networking to a tested small-game release. It retains the original module/lesson identities and Gate F. This expansion is an upper-undergraduate engineering volume within the wider undergraduate-to-master's self-study program.

There are 20 lessons, 20 full lab cards, 100 lesson quiz questions, 25 additional module-review questions and five integrating projects. Original questions 1-3 are preserved; questions 4-5 add application work. Attempt each quiz before opening the separate key. Passing a kernel is only one part of the full lab.

Read START_NEXT before beginning. Use LAB_WORKBOOK for implementation contracts and evidence. Each lesson below includes a teaching chapter, the original worked checkpoint and lab requirement, targeted external study, an engine-transfer task and a five-question quiz. Module reviews and full project specifications follow their four lessons.

The program's 420-650 planning-hour estimate includes practice and integration, not merely reading. Complete the assigned source study and independent implementations. No accredited degree or commercial-readiness claim is implied.

Module	Focus	Lessons
26	Collision detection and physical simulation	26.1-26.4
27	Animation, audio and interactive timing	27.1-27.4
28	Game AI, decision systems and machine learning	28.1-28.4
29	Multiplayer simulation and online services	29.1-29.4
30	Production quality and the undergraduate engine capstone	30.1-30.4

Evidence tiers

Algorithmic kernels, native offline tests, actual device/platform runs, process/socket integration and human observations are different evidence tiers. This package supplies the first two plus bounded companion simulations; its tests are not the student's capstone results. The full labs explicitly identify the additional comparison, device, transport and review work.

Module 26: Collision detection and physical simulation

Prerequisites: Modules 07-10, 14 and 24.

Original reading IDs: R28, R19, R23. See the reference directory.

Module goal: Build a deterministic test harness for a small physics scene, compare your simple implementation with a maintained physics library, and record precisely which physical and numerical assumptions differ.

Lesson 26.1: Broad phase, narrow phase and geometric robustness

Learning goal: Separate possible collisions from confirmed contacts.

The engineering problem

A projectile passes through a door only when the camera looks away. Before changing an epsilon, determine which subsystem owns collision membership and whether its update depends on rendering. Collision is simulation work. A visibility optimization must not remove an object from the collision world. Write the contract first: positions are meters, axes are right-handed in the simulation, AABBs are closed intervals, and normals point from body A toward body B.

Broad phase is a conservative filter

For n objects, exhaustive testing considers n(n-1)/2 unordered pairs. With 100 objects this is 4,950. Keep that implementation as an independent oracle. An optimized broad phase is correct when every truly overlapping pair is a candidate. It may return extra candidates. Comparing candidate sets for equality is therefore the wrong test; compare true contacts against the candidate superset, then compare narrow-phase results.

A uniform grid inserts each object's complete bounding box into every cell it overlaps. Compute a cell coordinate with floor(x/cell_size), not integer truncation toward zero. An object crossing the origin reveals the difference. Insert a closed box's maximum boundary conservatively even when it lies exactly on a cell edge. Emit canonical pairs (min_id, max_id), deduplicate them, and sort before the solver consumes them. Inserting only the center is not conservative for objects larger than a cell.

Grid performance depends on occupancy and object size. A crowded cell can still contain almost every pair. A huge object can occupy thousands of cells. Keep membership and pair counts in the profiler; a data structure name does not imply a speedup. A dynamic AABB tree is an alternative for heterogeneous sizes, while sweep-and-prune can exploit coherence along an axis. Box2D's collision documentation provides maintained shape queries and a dynamic-tree comparison target [S01].

Narrow phase and the meaning of contact

For circles, let d = center_B - center_A and R = radius_A + radius_B. Overlap under our closed-set policy means dot(d,d) <= R*R. Only after this decision do we need a square root for distance and a normalized contact normal. If the centers coincide, dividing by distance is invalid. Choose a deterministic fallback normal, such as the previous valid normal or a stable axis derived from IDs, and document it.

For AABBs, reject the pair when either axis separates the intervals. For convex polygons, the separating-axis theorem motivates projecting both polygons onto candidate edge normals: a separating interval proves no overlap. Absence of separation is not yet a complete contact manifold. Concave meshes need decomposition or a different representation. Do not present

the circle lab as a finished general-purpose collision library.

Worked trace: a cell boundary that breaks naive code

Let cell size be 1 m. Circle A has center (-0.1, 0), radius 0.2 m; circle B has center (0.15, 0), radius 0.1 m. Their center distance is 0.25 m and their radius sum is 0.3 m, so they overlap. A's x bounds are [-0.3, 0.1], spanning cells -1 and 0. B's bounds are [0.05, 0.25], in cell 0. Both also span y cells -1 and 0. A center-only or incorrectly truncated implementation can conceal a missing-cell defect in other placements. The robust implementation inserts full bounds and deduplicates shared-cell pairs.

Translate this entire scene by exactly 10 m and repeat. Geometry should be unchanged within the declared numeric tolerance. Repeat with a coincident center, an exact tangent and a radius near the minimum supported scale. These are different questions: boundary policy, degeneracy policy and scale policy must not be collapsed into one unexplained constant.

Implementation and debugging discipline

```
validate finite coordinates and positive cell size
for each stable body ID:
    compute shape AABB
    insert ID into every covered cell
for each occupied cell:
    emit all distinct ID pairs in canonical order
sort and unique candidates
run narrow phase; record contact IDs and normals
```

In a debug capture, retain bounds, cell memberships, candidate IDs and final contacts for the first missed pair. If a collision disappears, this trace identifies whether membership, filtering, geometry or lifetime failed. Reject invalid geometry before indexing it. Bound object size and cell coverage before allocating storage for user-authored scenes.

Deliberate practice

First solve the pair calculation on paper. Then implement the exercise kernel without reading its reference. The full lab adds a seeded randomized oracle comparison and an engine-owned spatial index. Measure uniform, clustered and one-giant-object scenes. A correct but slower grid is an acceptable finding; report it rather than choosing a convenient scene after seeing results.

Original worked checkpoint

For 100 objects, checking every unordered pair requires 100 x 99 / 2 = 4,950 pair tests. A broad phase helps only if its maintenance cost and candidate count are favorable for the scene.

Full practical assignment

Implement a grid broad phase and circle or box narrow phase. Pass: no true pairs are omitted compared with brute force over randomized scenes, and boundary-touching behavior is documented.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Conservative grid and circle contacts**. Run `starter/python/demos/lesson_26_1.py`; implement `starter/python/exercises/lesson_26_1.py`.

Assigned study and native transfer

R28 : collision module, shapes and broad-phase concepts; R19 : geometry and collision.

Read S01: shape primitives, geometric queries, contact manifolds and dynamic tree. Read the original R19 geometry material as a mathematical comparison. Record the exact library release used for any comparison.

Engine transfer: Port the broad-phase ownership and pair-order contract into the Volume 5 engine. Keep the brute-force oracle in a debug/headless target, independent of the grid implementation.

Check your understanding

1. May a correct broad phase return false positives?
2. How many unordered pairs exist for 100 objects?
3. Why document contact-normal orientation?
4. With 250 bodies, how many exhaustive unordered pairs are possible? What relationship must hold between true-contact and candidate sets?
5. A circle spans four cells but its center occupies one. Is center-only insertion sufficient? Give a repair and a duplicate-pair policy.

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 26.2: Rigid bodies, angular motion and collision impulses

Learning goal: Use conservation laws with explicit approximations.

State, units and approximation

A rigid body has a center-of-mass position, orientation, linear velocity, angular velocity, mass and inertia. A two-dimensional body has scalar angular velocity around the perpendicular axis. In three dimensions, angular velocity is a vector and inertia is a tensor whose frame matters. This lesson derives a 2D contact; the same conceptual separation between translation and rotation carries forward to 3D.

Force has units kg*m/s^2 and changes momentum over time. Impulse has units kg*m/s and changes velocity immediately in the model. Applying a force as an impulse every tick makes behavior depend on tick rate. Conversely, multiplying an already integrated impulse by dt weakens it incorrectly. Keep distinct API names: apply_force and apply_impulse. Static bodies use zero inverse mass and zero inverse inertia, not a floating-point infinity.

Deriving the contact impulse

Choose a unit normal n from A to B. Let r_A and r_B run from the bodies' centers of mass to the contact. The relative contact-point velocity is v_B + omega_B cross r_B - v_A - omega_A cross r_A. Its projection v_n on n is negative when the bodies approach under this convention. An impulse +j*n goes to B and -j*n to A. For a frictionless instantaneous collision:

```
k = inv_mass_A + inv_mass_B
    + (cross(r_A,n))^2 * inv_inertia_A
    + (cross(r_B,n))^2 * inv_inertia_B
j = -(1 + restitution) * v_n / k
```

Apply this only to an approaching contact with positive effective inverse mass k. For a separating contact, a positive bounce impulse would manufacture energy. The rotational terms explain why the same contact force can produce different linear outcomes at different lever arms. A normal is a direction, not a penetration distance; normalize it before using these equations.

For a center-of-mass one-dimensional collision, the rotational terms disappear. Equal masses m = 2 kg start at v_A = 3 m/s and v_B = 0. With restitution e = 0.5, relative velocity is -3 m/s, k = 1 kg^-1, and j = 4.5 N*s. The final velocities are 0.75 and 2.25 m/s. Momentum remains 6 kg*m/s. Initial kinetic energy is 9 J and final energy is 5.625 J. The energy loss is expected in this inelastic model, not a failed conservation test.

Worked rotational impulse

An isolated 2 kg body receives J = (6,0) N*s at offset r = (0,0.5) m. Its linear velocity changes by (3,0) m/s. Angular impulse is cross(r,J) = -3 kg*m^2/s. With inertia 0.25 kg*m^2, angular velocity changes by -12 rad/s. The sign follows the stated right-handed convention. If the application and renderer disagree about vertical-axis direction, the same physical clockwise motion may appear with an inverted screen angle; fix the transform boundary rather than negating impulses opportunistically.

Friction, resting contacts and energy

A tangential impulse opposes relative sliding. A simple Coulomb constraint bounds its magnitude by mu times the accumulated normal impulse. In an iterative solver, clamp the accumulated tangential impulse, then apply only the increment. Clamping each increment independently can exceed the intended total friction bound. Static and kinetic friction are richer than this compact approximation, especially around transitions and rolling contact.

Restitution is a collision-event model, not a demand that a resting stack bounce every tick. Many engines use a velocity threshold or solver-specific treatment. Study the selected library's contact and material behavior rather than assuming a universal parameter mapping [S02]. Gravity and the floor supply external forces and impulses, so momentum of only the falling body is not conserved. Select an isolated two-body scene when testing total momentum.

Failure analysis and integration

Check finite state, nonnegative inverse masses, supported restitution and inertia before applying impulses. A static-static contact has zero effective inverse mass and must not divide by zero. Distinguish a dynamic body temporarily at rest from a truly static body. Do not set inverse mass to zero merely because its current velocity is small.

The kernel lab handles center-of-mass velocity updates. The full lab must add the off-center case above, momentum and energy tables, an immovable wall, and tests across at least two fixed timesteps for sustained forces. Integrate through stable body handles. Queue creation and destruction outside solver iteration so callbacks cannot invalidate the contact list.

Original worked checkpoint

A 2 kg body receiving a 6 N*s impulse through its center changes velocity by 3 m/s. The same impulse away from its center also changes angular momentum.

Full practical assignment

Implement center-of-mass impulses and a simple rotational case. Pass: momentum is conserved in isolated tests within tolerance, static bodies remain fixed, and energy behavior matches the chosen restitution model.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **One-dimensional collision impulse**. Run `starter/python/demos/lesson_26_2.py`; implement `starter/python/exercises/lesson_26_2.py`.

Assigned study and native transfer

R28 : rigid bodies, contacts and simulation; R19 : physical simulation.

Read S02: rigid bodies, mass data, forces and impulses, friction and restitution. Re-derive the center-contact impulse without copying the supplied equation.

Engine transfer: Implement separate force and impulse commands in your engine. Add tests proving that the renderer cannot directly overwrite the physics-owned transform of a dynamic body.

Check your understanding

1. What velocity change follows from 6 N*s applied to 2 kg?
2. Why can an off-center impulse cause rotation?
3. Why use zero inverse mass for a static body?
4. Two 2 kg bodies have velocities 3 and 0 m/s and restitution 0.5. Compute impulse, final velocities and total final kinetic energy.
5. A 6 Ns *horizontal impulse acts 0.5 m above the center of a body with inertia 0.25 kg*m^2. What angular-velocity change results?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 26.3: Constraints, iterative solvers and continuous collision detection

Learning goal: Explain where discrete simulation fails and how solvers control error.

Two distinct sources of error

A projectile can miss a wall because collision was sampled only at endpoints. A chain can stretch because its constraint solver stopped before convergence. Increasing solver iterations addresses the second problem but does not necessarily discover the missing collision in the first. Diagnose detection, integration and constraint solving independently.

A distance constraint between particles A and B has residual C = length(x_B-x_A) - rest_length. A residual is a measured violation, not a visual impression. For a nonzero separation direction n and inverse masses w_A, w_B, a position-projection correction can distribute error in proportion to inverse mass:

```
x_A += n * C * w_A / (w_A + w_B)
x_B -= n * C * w_B / (w_A + w_B)
```

Skip an immovable pair and handle coincident points explicitly. A projection method is useful for learning convergence but is not identical to a velocity-level rigid-body solver. If corrected positions are converted to velocities, corrections can inject energy. Name the algorithm and measure the quantities it actually promises.

Iterative contact solving

A sequential impulse solver visits constraints, updates velocities locally, and repeats. Contact order affects intermediate results; warm starting reuses prior accumulated impulses to begin closer to a solution. For unilateral normal contact, accumulated normal impulse must remain nonnegative. Compute a proposed change, clamp the accumulated result, then apply only the difference. Otherwise the solver can pull bodies together with an attractive normal force.

Track the maximum and RMS residual across a three-link chain. Record initial positions, timestep, solver iteration count, ordering, gravity and tolerance. Compare 1, 2, 4, 8 and 16 iterations, then halve dt. A monotonic trend in one scene is evidence about that setup, not a guarantee for every contact graph. Box2D's documented sub-step count is not a drop-in synonym for every engine's solver-iteration parameter [S02].

Sweeping through time

At 30 m/s and 60 Hz, a projectile travels 0.5 m each tick. A 0.1 m wall can lie entirely between the starting and ending positions. For a point moving along x with constant velocity v toward a static slab [lo, hi], compute t0 = (lo-x)/v and t1 = (hi-x)/v, sort them, and intersect [t0,t1] with [0,dt]. Initial overlap has a declared hit time of zero in this teaching kernel. Zero velocity outside the slab means no hit. Zero velocity inside means initial overlap.

For an axis-aligned moving box, expand the obstacle by the box's half extents and sweep its center. In several dimensions, intersect the time intervals from each slab. For relative linear

motion of two objects, subtract velocities first. Rotation complicates swept bounds and contact geometry; a point-versus-slab result is not general rotational CCD. Library cast APIs may deliberately treat initial overlap differently, which must be recorded in the comparison [S01].

Worked time of impact

A point starts at x=0 with v=30 m/s; the wall occupies [0.24,0.26] m. Entry occurs at 0.24/30 = 0.008 s and exit at about 0.008667 s. Both occur before the 1/60 s tick ends. The endpoint at 0.5 m is outside the wall, but a sweep detects the entry. With a radius of 0.02 m, expand the wall to [0.22,0.28], so entry is about 0.007333 s.

After finding the first impact, advance to the impact time, solve the collision, and account for the remaining interval according to your policy. Repeated impacts can produce a near-zero-time loop. Bound the number of impacts per tick and specify a conservative fallback, such as stopping the fast body and emitting a diagnostic. Never let a malformed level create unbounded simulation work.

A disciplined experiment

Implement the swept kernel, then a chain residual experiment. The workbook requires both; a passing slab test does not complete the lesson. Keep raw residual tables rather than only a screenshot. Include a static pair, coincident anchors, a separating projectile, an initial overlap, and an impact exactly at dt. Measure whether more iterations or smaller dt delivers the better error/cost tradeoff in your chosen scene.

For the engine, CCD belongs to selected bodies and motion regimes. It is not a switch that cures all physics failures. Keep a record of which objects use sweeps, which use discrete contacts, and how impact events are scheduled relative to the presentation layer.

Original worked checkpoint

An object moving 30 m/s travels 0.5 m during a 1/60 s step. A wall much thinner than that can be crossed between sampled positions without either endpoint overlapping.

Full practical assignment

Create a fast-projectile tunneling test and a short chain of constraints. Pass: a continuous or swept test catches the projectile, and a plot or table shows constraint error versus iteration count and timestep.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Swept point versus one-dimensional slab**. Run `starter/python/demos/lesson_26_3.py`; implement `starter/python/exercises/lesson_26_3.py`.

Assigned study and native transfer

R28 : joints, solver settings and continuous collision; R19 : numerical simulation.

Read S01: ray casts, shape casts and time of impact. Read S02: stepping and joints. Read S19 for fixed-step accumulator behavior. Do not conflate substeps, solver iterations and render frames.

Engine transfer: Add a debug CCD flag, a per-tick impact cap and residual telemetry to your engine. Demonstrate that a dense contact scene cannot make one tick loop without a bound.

Check your understanding

1. How far does 30 m/s travel in 1/60 second?
2. Why can increasing solver iterations help?
3. Does CCD remove every numerical simulation problem?
4. A point starts at 0 m and moves at 30 m/s toward the slab [0.24,0.26] m during a 1/60 s tick. What is the entry time, and why does an endpoint test miss it?
5. A chain becomes less stretchy when solver iterations increase. Does this prove conservation of energy or solve projectile tunneling?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 26.4: Physics testing, determinism and engine integration

Learning goal: Validate simulation with invariants and adversarial scenes.

Define exactly what repeatable means

Repeatability is a claim with a scope: same executable and machine; same compiler settings on different machines; or bitwise agreement across named platforms. A visually similar replay is weaker than an identical serialized state. A hash proves agreement only for the bytes you included. Excluding the random generator, an AI cooldown or a spawn counter can make a misleadingly stable hash.

Write a state schema with stable entity IDs, transforms, velocities, gameplay fields, random state and pending deterministic commands. Exclude pointer addresses, padding, wall-clock timestamps and rendering interpolation. Define field order, integer widths, byte order and floating-point representation. Reject NaNs before hashing because NaNs have unusual comparison behavior and multiple bit patterns. Decide how negative zero is treated. Hashes localize differences; they do not make arithmetic deterministic.

The simulation boundary

Let S_t be the state before tick t. The transition consumes the input bundle I_t and produces S_(t+1) plus an event list E_t. Use an integer tick as the authoritative time index. An accumulator can run zero or more fixed steps per display frame, retaining a previous/current transform pair for rendering interpolation. Cap catch-up work and disclose the overload policy; silently discarding time changes behavior and must not be disguised as successful real-time operation [S19].

Stable iteration order matters when operations are not associative. Three floating-point values can sum differently under different grouping. Thread scheduling, unordered-container traversal and compiler contraction can therefore matter. The current Box2D simulation documentation describes explicit determinism strategies, including creation ordering and compiler assumptions, and warns that deterministic library behavior does not make an entire application deterministic [S02]. Do not repeat the outdated blanket claim that all physics-library cross-platform determinism is impossible.

Analytic checks expose integration error

For constant acceleration a, an exact one-dimensional reference is x(t)=x0+v0*t*+*0.5*a*t*t and v(t)=v0+a*t*. *Semi-implicit Euler updates velocity first and then position. After N equal steps of size h, position is x0+v0*N*h+0.5*a*h*h*N*(N+1). Its position error against the exact solution is 0.5*a*N*h*h, or 0.5*a*t*h. At a=-10 m/s^2, t=1 s and h=1/60 s, error is about -0.083333 m. Halving h should roughly halve this error for this experiment. The expected error is not zero.

A force-free body should maintain velocity. An isolated elastic two-body collision should conserve momentum and kinetic energy within tolerance. A damped system should not be judged by energy conservation. An engine under gravity should not be judged by constant

kinetic energy. Write the physical assumptions beside every invariant.

First-divergence debugging

Run a fixed input tape twice and compare state hashes at every tick. Deliberately alter one input at tick 37. The first differing post-step state should be S_38 under the naming convention above. If only the final hash is stored, hundreds of later contacts obscure the cause. Save the pre-divergence state, the input bundle, candidate/contact order, random state and build metadata. Then compare decoded fields, not just two opaque digest strings.

A serialization round trip is a separate test. Restore a state, advance the same next input, and compare against the unrestored run. Restoration that omits a warm-start cache may preserve visible transforms but change the next tick. Whether such caches belong in a rollback snapshot depends on the engine and the exact determinism goal.

Engine lifetime and fault policy

Do not destroy physics bodies while iterating the library's event buffer unless the API explicitly permits it. Convert events to stable engine handles, validate generations and enqueue commands for a known safe phase. Renderer interpolation reads snapshots rather than mutable solver storage. Entity deletion should clear presentation ownership without allowing a recycled ID to receive an old collision sound.

On a non-finite value, record the first responsible tick and operation, then stop the test or isolate the body under a documented runtime policy. Replacing every NaN with zero can hide a persistent defect and produce impossible rewards or movement. The full lab combines analytic tests, repeatability, serialization and an adversarial scene suite. This is the evidence foundation for later rollback networking, not an optional debugging convenience.

Original worked checkpoint

A freely moving body with no force should maintain velocity in the ideal model. A falling body before collision can be checked against an analytic constant-acceleration solution, separating integration error from collision behavior.

Full practical assignment

Build a regression suite with replay hashes on one pinned environment, finite-value checks and analytic tests. Pass: failures identify the first divergent tick and reports distinguish same-machine repeatability from cross-platform determinism.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Canonical replay and first divergence**. Run `starter/python/demos/lesson_26_4.py`; implement `starter/python/exercises/lesson_26_4.py`.

Assigned study and native transfer

R28 : simulation, determinism and limitations; revisit Modules 08 and 24.

Read S02: Determinism and simulation events. Read S19: fixed timestep and overload. Revisit original Modules 08 and 24 for numerical error and ownership.

Engine transfer: Publish a versioned engine state schema and first-divergence replay report. Classify the result as same-environment repeatability unless other named platform combinations were actually tested.

Check your understanding

1. Is same-machine repeatability proof of cross-platform determinism?
2. Why identify the first divergent tick?
3. What should happen when a simulation first creates a NaN?
4. Using semi-implicit Euler with acceleration -10 m/s^2, zero initial velocity, t=1 s and h=1/60 s, what is the position error against the analytic solution?
5. Two replay hashes agree, but an omitted AI cooldown differs. What has been proved, and what must change?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 26 project

Build a Physics Evidence Bench. Select a bounded scene containing two isolated colliding bodies, a static wall, one offset contact, a fast projectile and a short constrained chain. Keep scenes small enough to inspect every state transition. Use stable IDs, meters/seconds and a fixed update schedule. A graphical view is useful, but the headless state trace is the regression authority.

Integrate all four lessons: a conservative broad phase, narrow-phase contact convention, center/angular response, swept collision case and replay harness. Record the free-flight integrator separately from the contact solver. Run at least three timestep settings and several solver budgets without hiding energy injection or unresolved penetration. A limitation is acceptable when it is measured and bounded.

Compare at least one matched scene with a maintained library such as Box2D, using the exact library revision and settings. Do not substitute an online claim for running the comparison. The reference package includes a comparison template but does not download or execute that external library. No equality is promised when algorithms, skin widths or restitution thresholds differ.

Deliver scene files, seed/input tape, canonical state schema, invariant tests, first-divergence capture, raw measurements and a comparison report. Review asks: which invariants should hold here, what tolerances are numerical rather than geometric, and which differences are modeling choices?

Module 26 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. A projectile is missed only near cell boundaries. Which intermediate artifacts distinguish a broad-phase defect from a narrow-phase defect?
2. Why can an off-center contact require a smaller impulse than a center-only calculation for the same initial normal speed?
3. An engineer doubles solver iterations and calls it equivalent to halving the timestep. Explain the error.
4. What extra evidence is needed beyond matching two final-state hashes?
5. Your simple engine and a maintained library disagree on bounce height. What should be controlled before deciding either is wrong?

Module 27: Animation, audio and interactive timing

Prerequisites: Modules 07, 10, 18 and 24.

Original reading IDs: R19, R20, R22, R26. See the reference directory.

Module goal: Integrate an animated character or articulated object with event-driven audio. Demonstrate smooth interpolation, bounded audio work, adjustable sound categories and correct behavior when animation rate differs from simulation rate.

Lesson 27.1: Skeletons, transforms and skinning

Learning goal: Compute hierarchical animation without mixing coordinate spaces.

Coordinate spaces are part of the asset format

An articulated object is a hierarchy, not a bag of independent rotations. Choose column-vector notation for this lesson. A local joint transform maps from the joint's coordinates to its parent's coordinates. The world transform is parent_world * local. A root uses the skeleton's world transform as its parent. Multiplication order is a semantic contract, not a stylistic preference.

Evaluate ancestors before descendants. Imported node order is not necessarily parent-first, so either topologically sort the hierarchy or evaluate recursively with cycle detection and a depth bound. An invalid parent index, a cycle or an excessive depth is malformed asset data. Treat it as a load error rather than letting animation recurse until the stack fails.

Derive skinning from the bind pose

Let B_j be joint j's world transform in the bind pose and G_j its current world transform, all expressed in compatible spaces. To move a bind-space vertex through joint j, first map it into the joint's bind coordinates with inverse(B_j), then move it into the current pose with G_j. The resulting transform is G_j * inverse(B_j). If the renderer expects positions in mesh-local space, include inverse(mesh_world) on the left. Khronos's skinning tutorial and glTF specification make the joint and inverse-bind relationships explicit [S03, S04].

Linear blend skinning forms a weighted sum of transformed positions:

```
p_world = sum_j weight_j * G_j * inverse(B_j) * p_bind
sum_j weight_j = 1
```

In the bind pose, G_j=B_j, so each product is identity and the vertex reproduces its original position. This identity test is more valuable than immediately importing a detailed character. Validate finite, nonnegative weights, valid joint indices and a normalization policy. A zero total weight needs an explicit fallback or rejection; division by zero is not an acceptable repair.

Worked two-joint arm

Joint 0 is at the origin. Joint 1's local translation is (1,0), so its bind world position is (1,0). A vertex bound entirely to joint 1 lies at (2,0) in bind world coordinates. Its joint-local bind position is (1,0). Rotate joint 0 by 90 degrees while preserving joint 1's local translation. Joint 1 moves to (0,1), and the vertex moves to (0,2). Translating each joint independently without inheriting the parent rotation gives the wrong result.

For a blended vertex, use two weights of 0.5. At bind pose, both corrected transforms are identity, so blending still preserves the original vertex. Omitting inverse-bind correction adds the joint's bind placement again and produces displacement even before animation starts. This is a coordinate-space defect, not a bad blend curve.

Orientation and deformation

Interpolate translation and scale separately from rotation. A quaternion and its negative represent the same orientation; negate one endpoint when their dot product is negative before a shortest-path interpolation. Normalized linear interpolation is useful for small angular differences, while spherical interpolation better preserves constant angular speed between two orientations. Near-equal quaternions need a numerically stable branch. An unnormalized quaternion can introduce invalid transforms.

Linear blend skinning can collapse volume near twists. This is a known model limitation, not always a shader bug. For normals, the transform depends on the skinning and scaling model; nonuniform scale cannot be handled by blindly reusing a position transform. In the core lab, restrict joints to rigid transforms and document that restriction before adding more advanced deformation. Correctness within a declared model is preferable to silently accepting unsupported assets.

Asset and runtime ownership

Store the immutable hierarchy, bind transforms and clips as resources. Each character owns mutable playback time, blend state and evaluated pose buffers. Sharing a mutable pose across characters can make one actor's animation overwrite another's. The renderer receives a snapshot of joint matrices or a clearly synchronized buffer; it must not read a pose while a job is half finished.

The Python kernel uses 2D transforms to make the parent composition visible. The full lab adds a small skinned mesh or line skeleton to the engine, checks the bind identity, and displays local axes. Test a parent rotation, a translated mesh root, one invalid joint index and a rejected hierarchy cycle. Render at different rates without changing the authoritative animation time. Explain which transforms are local, skeleton-relative, mesh-relative and world-relative in the submission.

Original worked checkpoint

A vertex influenced equally by two joints has weights 0.5 and 0.5. Omitting the bind-pose correction can move it even when the skeleton is supposedly in its original pose.

Full practical assignment

Animate a two-joint mesh or line skeleton. Pass: the bind pose reproduces the source geometry, weights are validated and a parent rotation moves descendants consistently.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Parent-first two-dimensional pose evaluation**. Run `starter/python/demos/lesson_27_1.py`; implement `starter/python/exercises/lesson_27_1.py`.

Assigned study and native transfer

R19 : transformations and animation; R26 : skeletal animation concepts.

Read S03 in full, then S04 sections on skins and animations. Implement the two-joint identity test before loading complex art.

Engine transfer: Add an immutable skeleton resource and a per-instance pose buffer to the existing engine. Demonstrate two actors sharing one skeleton without sharing playback state.

Check your understanding

1. Why evaluate parent transforms before dependent children?
2. What should ordinary normalized skinning weights sum to?
3. Why avoid directly interpolating arbitrary rotation-matrix entries?
4. A root rotates 90 degrees and its child has local translation (1,0). Where is the child in world coordinates under column-vector parent*local composition?
5. Why should G_j * inverse(B_j) be identity in the bind pose, and what extra transform is needed for a mesh-local output buffer?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 27.2: Animation blending, state machines and inverse kinematics

Learning goal: Coordinate authored motion with runtime goals.

Animation is a controlled state transition

A character does not simply play whichever clip was requested last. Define an animation state machine with entry behavior, exit behavior, interruption rules and transition duration. Idle, walk and action states are enough for this lab. Specify whether an action can be interrupted by movement, damage or pause, and whether a repeated trigger restarts it or is ignored.

Separate state selection from pose evaluation. A transition blends sampled poses, but the gameplay action may have an exact authoritative start tick. The blend's visual midpoint is not automatically the moment a hit occurs. Blend translation and rotation using the conventions from Lesson 27.1. A linear blend parameter rises from 0 to 1 over a declared duration; a smooth curve changes derivative continuity, not the underlying event ownership.

Phase and root motion

Two walk clips can have compatible speeds but incompatible foot phases. Blending left-foot contact with right-foot contact can produce sliding. Normalize cycle phase or use markers when the asset design requires it. Clip time must have a unit: seconds, normalized cycle fraction or ticks. A variable called time without a unit is a common source of integration mistakes.

Root motion is displacement authored by animation. Physics-driven movement instead derives displacement from simulation. Choose one authority. A useful contract is that animation proposes a displacement, the movement controller resolves collision and accepts a corrected displacement, and the visual pose adapts to the result. Applying both the full authored displacement and the physics velocity double-counts movement. Teleporting a dynamic body to every animation root sample bypasses its physical constraints.

Derive a two-link inverse-kinematics solution

For a planar arm with lengths L1 and L2 and target (x,y), let d=sqrt(xx+yy). The reachable radial range without joint-angle limits is abs(L1-L2) <= d <= L1+L2. The law of cosines gives an elbow angle theta2 through:

```
c2 = (d*d - L1*L1 - L2*L2) / (2*L1*L2)
theta2 = acos(clamp(c2, -1, 1))
theta1 = atan2(y,x)
         - atan2(L2*sin(theta2), L1+L2*cos(theta2))
```

The negative theta2 branch gives the alternate elbow configuration. A pole direction, previous pose or joint-limit policy selects a branch. Clamp unreachable targets to the reach interval and report that clamping occurred. The clamp on c2 handles small rounding excursions, but it must not conceal an invalid negative link length.

Worked target and singularity

With L1=L2=1 and target (1,1), d squared is 2, c2=0, theta2=90 degrees and theta1=0 degrees for the chosen branch. Forward kinematics returns (1,1). For target (3,0), the maximum reach is 2, so the arm can extend toward (2,0) but cannot reach (3,0). Reporting a residual of 1 is honest; repeatedly iterating toward an impossible target is wasted computation.

At the origin with equal links, a folded arm can reach the target but the shoulder direction is not uniquely determined. At full extension, small target changes can create large angular changes. Preserve a stable prior direction or use an explicit fallback. In 3D, singularities, joint limits and multiple chains add complexity; this analytic 2D solver is a teaching model rather than a general full-body IK system.

Ordering the animation pipeline

A typical bounded pipeline samples base clips, blends them, applies an additive layer, solves selected IK constraints, then computes final world transforms. The exact ordering matters. Solving foot IK before an unrelated later layer moves the pelvis can undo the contact. Recompute dependent transforms after local changes. Cache immutable clip data, not partially evaluated mutable poses shared across actors.

Trace each transition with source state, target state, trigger, start tick, blend duration and interruption reason. When a character freezes, the trace should reveal whether a state never exited, a clip time stopped or an IK target became invalid. Do not debug all three through a single final mesh screenshot.

Deliberate practice

The exercise kernel computes the two-link solution and reports reachability. The full lab must also implement idle/walk/action transitions, cancellation and root-motion ownership. Test target motion across the reach boundary, a pause during a blend, a movement trigger during an action and two consecutive interruptions. Verify pose continuity numerically and inspect behavior at multiple rendering rates. A correct endpoint equation alone does not complete an animation controller.

Original worked checkpoint

A two-link arm with lengths 1 and 1 cannot reach a target 3 units from its base. A solver should clamp or report the unreachable condition rather than generating invalid angles.

Full practical assignment

Build idle/walk/action transitions and a two-link IK demonstration. Pass: interruptions behave as specified, unreachable targets are handled, and root-motion ownership is documented.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Analytic two-link inverse kinematics**. Run `starter/python/demos/lesson_27_2.py`; implement `starter/python/exercises/lesson_27_2.py`.

Assigned study and native transfer

R19 : animation and inverse kinematics; R26 : animation blueprints and blending concepts, compared with your own small implementation.

Read S04 animation interpolation and node transforms. Revisit R19 animation and R26 animation-system concepts. Derive the two-link geometry from the law of cosines.

Engine transfer: Implement the state/pose separation in your engine and make the animation trace visible in the inspector. Add one cancellation test that frees an action-owned resource.

Check your understanding

1. Can every IK target be reached?
2. What is the maximum reach of two collinear links of length 1 each?
3. Why define root-motion ownership?
4. For links of length 1 and 1 and target (1,1), compute one elbow and shoulder solution. Verify it with forward kinematics.
5. An action clip moves the root 0.3 m while the physics controller also moves it 0.3 m in the same tick. What contract is missing?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 27.3: Digital audio, mixing and real-time callback discipline

Learning goal: Generate sound without missing the output deadline.

Audio runs on a different deadline

A sample is one channel's value at one instant. An audio frame contains one sample for every channel. At 48,000 frames per second, 256 stereo frames contain 512 sample values but span only 256/48000 = 0.005333 seconds. Confusing samples, frames and bytes can halve a buffer or overrun it. State the format: sample rate, channel count, sample type and interleaving.

A game may tolerate an occasional long visual frame; the audio device still needs its next buffer. Disk access, memory allocation, waiting on a mutex, console logging and unpredictable callbacks can exceed the available time. PortAudio's callback guidance explicitly cautions against blocking and unpredictable work [S05]. A callback's practical deadline also depends on the platform and buffering arrangement; the buffer duration is a useful budget calculation, not proof of end-to-end latency.

Generate and mix a signal

A sine oscillator advances phase by 2*pi*frequency/sample_rate each frame. Preserve phase between callbacks or each buffer boundary can introduce a discontinuity. For a mono mix, y[n] = sum_i gain_i*x_i[n]. A conservative no-clipping policy for sources bounded in [-1,1] is sum_i abs(gain_i) <= 1. Two voices at gains 0.25 each have a worst-case peak of 0.5, even when their phases align.

A limiter or soft clipper is a different policy and changes the waveform. Do not call saturation distortion-free. Abrupt gain or playback changes can create clicks; use a short ramp with a specified duration. Choose frequencies below the Nyquist limit, but remember that discontinuous or nonlinear waveforms can introduce harmonics above it. The core lab uses generated tones so no third-party assets or codecs are needed.

Bounded work and ownership

Keep a fixed-size voice array. A callback processes at most a declared number of control commands and at most the configured voice count times the output frame count. Even a lock-free queue can be abused by draining until empty while the producer keeps filling it. Bound commands per callback. If the queue fills, drop or coalesce noncritical updates under an explicit policy and count the event outside the callback.

A single-producer/single-consumer ring has one writer for its write index and one writer for its read index. The producer writes a slot before publishing the new write index with release ordering. The consumer observes that index with acquire ordering before reading the slot. Slot reuse requires the reverse publication of the read index. This ownership pattern does not extend automatically to multiple producers. The native C++ companion makes these boundaries concrete and checks whether its atomic index type is lock-free on the build target.

Allocate and decode before playback. Pass stable handles or immutable sample buffers to audio, not pointers to resources that the game thread may immediately free. A stop command must be acknowledged or otherwise synchronized before resource reclamation. An audio callback should not depend on a renderer lock or gameplay allocation to finish.

Worked budget and stress test

Suppose the chosen device consumes 256-frame buffers at 48 kHz. An illustrative measured callback cost of 0.7 ms leaves about 4.63 ms relative to the buffer duration. That is not a guarantee against scheduling delays. Record maximum observed callback time, distribution, underrun count if exposed, voice count and the measurement method. Never invent an underrun count when the API does not expose one; report the available observation and its limitation.

The native harness times offline callback invocations, so it can test arithmetic, bounded command processing and queue behavior. It does not prove that a real device is underrun-free. The full lesson requires the learner to connect a maintained platform API, mix two tones, vary gains and run a deadline-stress test. SDL3 stream callbacks have their own thread and API contracts; consult the selected SDL3 page rather than copying SDL2 callback code [S06].

Completion boundary

The Python exercise creates offline samples to verify phase continuity and gain bounds. Python execution is not presented as a real-time audio callback implementation. The C++ harness demonstrates a native ownership pattern without linking a device library. Actual hardware playback, mute/device-loss behavior and callback stress evidence remain required lab work. Start at low output volume and avoid testing with unexpectedly loud peaks. Preserve separate music/effects settings and a master mute for the next lesson.

Original worked checkpoint

A 256-frame buffer at 48,000 frames per second spans 256/48,000 seconds, about 5.33 ms. A callback that blocks for 20 ms can miss that timing requirement.

Full practical assignment

Mix two generated tones with adjustable gains using a platform audio API. Pass: no clipping under the documented gain policy, bounded callback work and a recorded underrun or deadline-stress test.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Offline phase-continuous tone mixer**. Run `starter/python/demos/lesson_27_3.py`; implement `starter/python/exercises/lesson_27_3.py`.

Assigned study and native transfer

R10 from Volume 2: sampling and filtering; R22 : SDL3 audio; R20 : audio systems.

Read S05 callback guidance and S06 SDL3 stream-callback contract. Revisit Module 18 for memory ordering. Record exact device/API format and version in your lab report.

Engine transfer: Connect the native mixer pattern to your engine audio adapter. Keep device-specific code outside the simulation and demonstrate graceful operation when no output device exists.

Check your understanding

1. What duration does 256 frames represent at 48 kHz?
2. Why avoid disk reads inside an audio callback?
3. Does halving the buffer automatically improve every aspect of audio?
4. How many sample values are in 256 stereo frames, and what duration do they represent at 48 kHz?
5. Why can draining a lock-free command queue until empty still violate a callback deadline? Give a bounded alternative.

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 27.4: Spatial audio, animation events and multimodal accessibility

Learning goal: Synchronize feedback while preserving user control.

A game event is not an audio call

The simulation decides that a relay activated. Presentation decides how that fact is conveyed: a tone, an on-screen message, an animation or a vibration where supported. Keep the authoritative event separate from any one output channel. Otherwise muting sound may accidentally disable gameplay logic, or replaying physics may trigger a second reward.

Use an event identity such as (match_epoch, tick, entity_id, local_event_index). The identity must be stable when the same deterministic tick is replayed. A pointer or wall-clock timestamp is unsuitable. Store presentation deduplication separately from rollback state so restoring a simulation snapshot does not erase the memory of already-played cues. Bound the deduplication window by the rollback horizon plus a safety margin, and reset it when the match epoch changes.

Event policy under prediction

Some predicted effects can happen immediately and be corrected later; others should wait for confirmation. A footstep can be deduplicated with low consequence. A purchased reward must be durably confirmed and cannot be treated as an audio-style best effort. If a predicted cue turns out not to have happened, the application cannot unplay the sound. This is a policy tradeoff, not an algorithm that eliminates all perceptual error.

Animation markers need similar identity. If a clip crosses a marker while the renderer skips a frame, the marker should still fire based on the traversed time interval. If playback loops, include a cycle index. Pausing and seeking require explicit marker semantics. Firing a marker whenever the current time is greater than its threshold causes it to fire every subsequent frame.

Derive a simple spatial pan

Stereo panning is a teaching approximation, not a full head-related transfer function. Let p range from -1 for left to +1 for right. Equal-power gains can be defined by angle=(p+1)*pi/4, left=cos(angle), right=sin(angle). The sum of squared gains is one. At center, each gain is about 0.7071, not 0.5. This preserves the mathematical power sum for an isolated signal; perceived loudness still depends on content and playback conditions.

Distance attenuation is another independent mapping. Choose a bounded rule, such as gain=1/(1+(distance/reference_distance)^2), and state that it is a game-design curve rather than a complete acoustic model. An occlusion test can drive filtering, but avoid running expensive geometry queries inside the audio callback. The game thread sends a bounded parameter update instead.

Worked multi-clock trace

A simulation runs at 60 Hz. Tick 90 spans authoritative time 1.5 to about 1.5167 s. A foot-contact marker generates event (epoch7,90,actor12,0). The display renders twice before the next simulation step, but both frames observe the same event identity, so it is presented once. A later correction restores tick 88 and replays through tick 90. The event is regenerated with the same identity and is not presented again.

If the restored simulation produces a different contact at tick 90, its deterministic local index and event type must follow the schema. Never deduplicate solely by tick: multiple legitimate events can occur in one tick. Never allow an unbounded global set to retain every event from a multi-hour session.

Accessibility is part of the event contract

A critical audio cue needs another usable channel. For the relay game, activation can produce a visible status change and readable text in addition to a tone. Color alone should not carry the entire distinction; use a shape, icon or label as well. Music, effects, speech and master volume have different user purposes. Muting one category must not suppress a necessary visual cue. Microsoft XAG 103 addresses additional channels for critical information [S23].

Test the complete task with output muted, without relying on a tester remembering where the cue would have been. Test whether captions describe important non-speech events, whether text persists long enough and whether simultaneous cues remain understandable. A guideline review helps identify barriers but does not replace user evaluation or establish legal compliance [S07].

Deliberate practice

Implement event deduplication and the equal-power pan kernel. Then connect events to separate audio and visual adapters. Record a replay trace containing a repeated event, a second legitimate event at the same tick and a new match epoch. The full lab adds separate category controls, a bounded event cache and a muted-audio task test. Do not let the success of a deduplication unit test stand in for actual accessible presentation.

Original worked checkpoint

A predicted footstep may be encountered again during replay. Assigning an event identity and a presentation policy prevents an unintended duplicate sound while preserving the corrected simulation.

Full practical assignment

Add spatial panning, separate music/effects controls and a visual equivalent for a critical audio cue. Pass: replay does not duplicate one-shot events and a muted-audio test still allows the required task.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Equal-power pan and stable presentation events**. Run `starter/python/demos/lesson_27_4.py`; implement

`starter/python/exercises/lesson_27_4.py`.

Assigned study and native transfer

R20 : audio and event systems; R22 : audio interfaces; R29 from the resource directory: audio and subtitle accessibility.

Read S23 additional channels and S07 guideline scope. Revisit S14 for rollback context. Review the original R22/R26 event and audio integration material.

Engine transfer: Route your engine events into interchangeable audio, visual and optional haptic adapters. Demonstrate one critical task with audio muted and no dependence on color alone.

Check your understanding

1. Is stereo panning a complete spatial-hearing model?
2. Why give presentation events stable identities during replay?
3. Why provide a non-audio equivalent for a critical cue?
4. At center pan under the equal-power rule, what are the left and right gains? What quantity is preserved?
5. A rollback snapshot restores the set of already-played sounds to an earlier version. What bug follows, and where should this set live?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 27 project

Build an Articulated Signal Station. A two-link arm or small character can idle, move and perform one interrupted action. Its motion must have a defined hierarchy, bind pose and blend policy. One semantic gameplay event drives an audio cue and a visual alternative. The example theme is optional; an original equivalent is welcome.

Prepare the native audio resources outside the callback, route bounded commands through an ownership-safe channel and keep music/effects controls independent. Implement the actual platform audio path and record a stress test. The supplied offline harness establishes only the behavior it executes; it does not establish the student’s device deadline or the absence of dropouts.

Run the simulation at a fixed step and show presentation at two rates. Roll back across an already presented event and demonstrate that a one-shot is not duplicated. Complete a muted-audio user task and record where the tester hesitated. Do not claim broad accessibility from a single participant.

Deliver transform/space definitions, action state table, IK tests, callback resource/work caps, raw device observations, event identity/retention policy, settings save/load and the

muted-task record. The reviewer should be able to trigger cancellation and identify which system owns motion, sound emission and cleanup.

Module 27 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. A bind-pose mesh moves when skinning is enabled. Give a structured diagnosis.
2. An action is canceled during a blend but its attack sound still fires. What ownership rules are missing?
3. A callback has a lock-free queue but misses deadlines. Give three plausible reasons.
4. Why should the presentation-event deduplication ledger not be restored with the rolled-back simulation state?
5. What makes a critical visual audio alternative successful rather than merely present?

Module 28: Game AI, decision systems and machine learning

Prerequisites: Modules 09, 14, 24 and 26.

Original reading IDs: R20, R46, R47. See the reference directory.

Module goal: Build an inspectable agent that navigates, chooses actions and recovers from blocked plans. Compare a rules-based baseline with one small learned or search-based component; evaluate behavior and runtime cost, not only win rate.

Lesson 28.1: Navigation graphs, steering and dynamic obstacles

Learning goal: Separate finding a route from executing movement.

Planning and movement solve different problems

Pathfinding answers which route connects a start and a goal in a model. Steering answers how an actor moves along that route under speed, acceleration and collision constraints. A perfect route through point-sized graph nodes can be impossible for a wide character. A local steering controller can avoid a nearby obstacle yet fail to discover a route around a large wall. Keep both layers inspectable.

Define a navigation state: grid cell, graph vertex or polygon. Define legal transitions and their costs. For the core grid, movement is four-connected with unit cost; Manhattan distance is an admissible and consistent heuristic. Adding diagonal moves, terrain multipliers or teleport edges changes the assumptions. A heuristic that overestimates can sacrifice shortest-path guarantees. Berkeley's informed-search material is the assigned algorithm reference [S08].

A* with an explicit budget

A* orders the frontier by f=g+h, where g is the best known cost from the start and h estimates remaining cost. Maintain a best-cost map and predecessor map. A heap can contain stale entries after a better route is discovered; skip an entry whose g no longer matches the best known cost. Stable tie-breaking makes traces easier to reproduce. If using a merely admissible but inconsistent heuristic, a graph-search implementation may need node reopening; do not assume a closed set is always safe.

A node-expansion budget is reproducible and useful for teaching, but it is not a wall-clock guarantee if one expansion has unbounded neighbor work. Bound degree and data sizes too. Return a distinct status for found, unreachable and budget_exhausted. Confusing an interrupted search with a proof of no path produces bad gameplay decisions. An incremental planner may retain frontier state across ticks; a simpler lab can stop and use a safe idle action.

Geometry belongs in the navigation model

A 1.0 m wide agent cannot pass a 0.8 m doorway under a no-overlap model. Inflate obstacles by the agent radius or erode walkable space by its footprint. Different agent sizes may need different navigation data or clearance tests. For a grid, make clear whether blocked cells represent centers or occupied areas. A path's graph validity does not prove that every smoothed segment has geometric clearance.

Dynamic obstacles create a versioning problem. Tag the path with the navigation revision and revalidate upcoming segments. Replanning every frame for every unit can overwhelm an RTS. Use blocked-path detection, staggered work, a global expansion budget and a queue with explicit priority. Avoid starvation by recording wait time, not only distance to the camera.

Worked path and budget

On an empty four-connected grid, start (0,0) and goal (3,2) have Manhattan distance 5. Every shortest path has five moves. If a wall blocks (1,0), (1,1) and (1,2), a route may need to go around it if the grid extends far enough; if the grid is only three rows high and the wall spans its full height, no route exists. A budget of two expansions can stop before either result is known. Report budget_exhausted rather than unreachable.

For steering, let desired velocity point toward the next waypoint at a capped speed. Limit velocity change by max_acceleration*dt. Advance the body through collision resolution rather than assigning the target position directly. An arrival radius avoids oscillation around an exact point, but a radius larger than a narrow doorway can skip an essential waypoint. Match steering tolerances to geometry and speed.

A useful debug view

Display blocked cells, agent footprint, planned path, current waypoint, desired velocity, actual velocity and search status. Log when a path becomes invalid and why. A stationary unit can be stuck because the planner returned no path, the steering acceleration is zero, collision rejects motion or a behavior action was canceled. One final position does not distinguish those failures.

The exercise kernel implements bounded A* over a supplied grid. Its reference is intentionally small. The full lab adds footprint clearance, steering, a dynamic obstacle and an unreachable-goal test. Measure expansions and elapsed time separately. Compare the A* path cost to a breadth-first or Dijkstra oracle on small maps, not to the same A* implementation with a different display setting.

Engine integration

Navigation workers consume an immutable world snapshot or versioned query interface. They return a path plus the revision used to compute it. Before installing the result, check that the requesting entity still exists and that its goal generation matches. Otherwise an old asynchronous result can steer a new entity that reused the same index. This is the same lifetime discipline introduced for physics and animation resources.

Original worked checkpoint

A doorway 0.8 units wide is not traversable by a circular agent with diameter 1.0 under a no-overlap model, even if the navigation graph contains an edge through its center.

Full practical assignment

Combine A* with a steering controller and a blocked-path detector. Pass: agent size is respected, unreachable goals terminate or report failure, and repeated replanning has a bounded budget.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Budgeted four-connected A-star**. Run `starter/python/demos/lesson_28_1.py`; implement `starter/python/exercises/lesson_28_1.py`.

Assigned study and native transfer

R46 : Search; R20 : game AI and navigation concepts.

Read S08 on A*, admissibility and consistency. Revisit Modules 09 and 14 for graphs, heaps and complexity. Compare one tiny map against an independent shortest-path oracle.

Engine transfer: Add a globally budgeted navigation queue and stale-result rejection to the engine. Demonstrate a wide and a narrow actor receiving different valid routes.

Check your understanding

1. How do pathfinding and steering differ?
2. Can the example 1.0-wide agent pass through the 0.8-wide doorway?
3. Why bound replanning work?
4. For a four-connected unit-cost grid, what is the Manhattan heuristic from (0,0) to (3,2)? Does a two-expansion budget prove a route is impossible?
5. A smoothed path segment cuts across an obstacle corner although every original grid node is free. What validation is missing?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 28.2: Behavior trees, utility systems and explainable decisions

Learning goal: Represent behavior so designers can understand its failures.

Make the reason for a decision observable

An AI that looks intelligent in one recording may be impossible to maintain. Designers need to know which condition selected an action, what data it used and why the previous action stopped. Represent perception, decision and execution separately. A sensor updates facts; a selector chooses an action; the action owns multi-tick work and cleanup.

A finite-state machine is often sufficient for a small game. A behavior tree composes conditions and actions. A sequence generally advances while children succeed and returns on failure or running; a selector generally tries alternatives until one succeeds or runs. Reactive and memory variants differ in whether they revisit earlier children. Those semantics must be explicit. BehaviorTree.CPP's own documentation emphasizes nonblocking actions and state-transition logging [S09]. The core lesson does not require that library.

Running is a real state

An action that requests a path cannot report success merely because it started. It remains running until the request completes and the destination is reached, or fails when the goal becomes invalid. Cancellation must release its reservations, unsubscribe callbacks and invalidate outstanding generations. A canceled action that later receives a path result must not resurrect itself.

Define a lifecycle: enter, tick, success/failure, and cancel/exit. An action should release a resource exactly once even if cancellation races conceptually with completion. Use ownership tokens or generation checks rather than loosely paired booleans. Keep the decision trace bounded; a ring of the last few hundred records is more useful than an unbounded log in a shipping build.

Utility scores and hysteresis

A utility system computes comparable scores for candidate actions. Normalize feature ranges and document their meaning. Mixing health on a 0..100 scale with distance on a 0..1 scale gives arbitrary weight to units. An action may be invalid regardless of its score; filter invalid candidates before selection.

Pure argmax can oscillate when scores are nearly tied. Introduce a switching margin, a minimum commitment time or both. For a margin m, retain the current valid action unless best_score > current_score + m. Emergency actions can bypass ordinary commitment. Hysteresis trades immediate responsiveness for stability; a margin is not a universal solution and must be tuned against actual transitions.

Worked selector trace

Suppose attack has utility 0.61, retreat 0.60 and patrol 0.20. The actor is retreating and the switch margin is 0.05. Attack does not exceed retreat by more than 0.05, so retreat remains active. On the next tick, health falls and retreat rises to 0.90; the selector still retains retreat for a different reason. Later attack rises to 0.70 while retreat falls to 0.50, so switching is permitted. Record scores, validity, current action, chosen action and the reason.

If the current action becomes invalid, do not retain it merely because the margin is large. With no valid actions, choose a safe idle/failure state and emit a diagnostic. Treat NaN utility as invalid input rather than letting language-specific comparison behavior select a mysterious winner. Stable action IDs break exact ties deterministically.

Blackboard discipline

A blackboard stores shared facts, but an unrestricted global dictionary can hide dependencies. Give entries types, owners and freshness. A target position from five seconds ago is different from a current observation. A last-seen target can be a deliberate memory feature, but label its timestamp and uncertainty. Do not let AI read hidden authoritative enemy state that its game design says it cannot know.

For concurrent jobs, decisions should consume a coherent snapshot. A mix of this tick's health and last tick's target ownership can create impossible combinations. Limit mutation to a known phase or version the input facts. Log the fact revision with the decision so failures can be replayed.

Deliberate practice

The exercise implements a deterministic utility selector with hysteresis. The full lab adds at least one multi-tick action and three induced failures: unreachable target, canceled resource owner and missing/stale perception. Explain each failure through a trace, not a print statement saying only 'AI failed.' Test that cancellation frees its resource and that an old callback cannot re-enter the action.

For the capstone, prefer one inspectable behavior that reliably interacts with the finished gameplay loop over many opaque behaviors. A rules-based patrol/repair/retreat controller can be excellent engineering evidence. Machine learning in the next lesson is an alternative tool, not a requirement to replace understandable decisions everywhere.

Original worked checkpoint

An agent choosing between attack score 0.51 and retreat score 0.50 can oscillate as small observations change. A switching threshold or minimum commitment interval can stabilize the policy, with explicit tradeoffs.

Full practical assignment

Implement either a small behavior tree or utility selector with a trace viewer. Pass: action cancellation releases resources and three induced failure cases produce understandable explanations.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Deterministic utility selection with hysteresis**. Run `starter/python/demos/lesson_28_2.py`; implement `starter/python/exercises/lesson_28_2.py`.

Assigned study and native transfer

R20 : game AI; R46 : agents, utilities and decision-making foundations.

Read S09 and its linked basic behavior-tree concepts. Compare sequence/selector semantics with your own explicit lifecycle. Revisit Module 24 ownership and event queues.

Engine transfer: Expose a bounded decision trace in the engine inspector, including facts, scores, chosen action, cancellation reason and outstanding request generation.

Check your understanding

1. What statuses should a multi-frame action expose?
2. Why can nearly tied utility scores cause oscillation?
3. Why is a decision trace valuable?
4. The current action scores 0.60, the best alternative scores 0.64 and the switching margin is 0.05. Which action remains, assuming both are valid?
5. A canceled navigation action receives its asynchronous result later and starts moving again. What state or ownership check should prevent this?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 28.3: Adversarial search, planning and simulation budgets

Learning goal: Trade decision quality against bounded computation.

Search is a budgeted policy, not a magical opponent

For a deterministic, turn-taking, zero-sum, perfect-information game, minimax chooses the move with the best outcome against an optimal opponent. At maximizing nodes, take the largest child value; at minimizing nodes, take the smallest. Terminal values are game-defined outcomes. Depth-limited evaluation estimates nonterminal states and therefore changes the strength of the guarantee. Berkeley's minimax chapter is the formal reading [S10].

An RTS is often simultaneous, partially observed and much larger than the tiny turn-taking model. A small search can still help choose a tactical action, but state clearly what the internal model omits. Do not claim that solving a toy minimax tree solves full RTS strategy. The capstone needs one bounded, inspectable decision system, not an unrestricted strategist.

Alpha-beta preserves the search objective

Alpha is a lower bound already available to the maximizer; beta is an upper bound already available to the minimizer. When a branch cannot improve the ancestor's choice, stop exploring it. With correct implementation and the same depth/evaluation model, alpha-beta returns the same minimax value while potentially visiting fewer nodes. Move ordering affects how much pruning occurs, not which fully searched objective is being optimized.

For a root maximizer whose left minimizing child has leaves [3,5], the left value is 3. Now inspect a right minimizing child with first leaf 2. Its final value cannot exceed 2, so the root already prefers the left branch and the remaining right leaves can be pruned. This is a proof about bounds, not a heuristic guess that the remaining leaves are probably uninteresting.

Exponential growth and iterative deepening

A branching factor of 10 at depth 6 gives about one million leaf paths, before counting internal nodes. A time budget alone is hard to reproduce across machines. Record both elapsed time and expanded nodes. In the teaching harness, a node budget ensures a bounded search workload given bounded move generation. In a runtime system, also bound per-node operations and check a deadline at useful intervals.

Iterative deepening completes depth 1, then 2, and so on. Retain the last fully completed root result. If depth 4 is interrupted, do not label its partly evaluated root as equivalent to a completed depth-4 search. Before searching, choose a legal fallback action so the actor always has a valid response. A zero budget should return that fallback, not an uninitialized move index.

Worked budget decision

A small game offers moves 'take one' and 'take two' from a pile. Taking the last item wins. With a pile of 4, taking one leaves 3, which is losing for an optimal opponent in this rule set. A complete search can prove the move. With a budget too small to finish any root depth, the system returns its declared legal fallback. The fallback may be strategically poor, but it is not a correctness failure unless the API promised optimality under that budget.

Separate the result fields: chosen_move, completed_depth, expanded_nodes, budget_exhausted and value_if_complete. That distinction makes performance-quality experiments meaningful. A result that silently replaces 'unknown' with zero can bias action selection toward branches that happened to time out.

Planning alternatives and evaluation

Monte Carlo tree search estimates action quality through simulations; it also needs rollout, exploration and budget policies. A planner with action preconditions and effects may fit a resource-management problem better than minimax. Choose the model that matches the decision, and compare to a simple baseline. A deeper but inaccurate internal simulation can be worse than a shallow model with better assumptions.

Test multiple seeds, starting states and opponents. A single self-play win rate can reward shared bugs or a narrow strategy. Count illegal actions, timeouts, nodes, latency and outcome quality separately. Keep a deterministic tie-break so regressions can be traced. When an evaluation function changes, preserve the test corpus rather than choosing new easy positions.

Deliberate practice

The kernel operates on a small explicit minimax tree with alpha-beta and a strict node cap. The full lab builds a tiny legal-move game, adds iterative deepening and a fallback, and compares against an exhaustive solver for small states. Induce a zero budget, a terminal root, an exact tie and an interrupted deeper iteration. For an engine transfer, demonstrate that a decision deadline cannot stall the simulation indefinitely and that the trace reports incomplete search honestly.

Original worked checkpoint

A branching factor of 10 searched to depth 6 can produce roughly one million leaf paths before pruning. A 2 ms decision budget cannot be assumed sufficient without measurement and a bounded policy.

Full practical assignment

Implement minimax with alpha-beta or a small MCTS agent for a tiny game. Pass: legal moves are always returned within a defined budget or fallback, and evaluation includes multiple seeds and opponents.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Node-bounded alpha-beta value search**. Run

`starter/python/demos/lesson_28_3.py`; implement `starter/python/exercises/lesson_28_3.py`.

Assigned study and native transfer

R46 : Games, Minimax, Expectimax and Monte Carlo Tree Search.

Read S10 on minimax, alpha-beta and evaluation functions. Derive one pruning decision by hand. Use the original Module 14 complexity tools to bound your move generator.

Engine transfer: Integrate a decision job with a legal fallback and explicit completion status. Record how increasing the budget changes both response latency and decision quality.

Check your understanding

1. Does alpha-beta change the minimax objective when applied correctly?
2. How many depth-6 paths arise from branching factor 10 without pruning?
3. Why use a fallback action?
4. A root maximizer has already found value 3. A new minimizing child first reveals leaf value 2. Why may its remaining leaves be pruned?
5. Depth 3 completed but depth 4 exhausts its node budget halfway through the root. Which result should iterative deepening return, and what should be reported?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 28.4: Learning systems, neural inference and responsible evaluation

Learning goal: Understand learning fundamentals without replacing every subsystem with a model.

Start with a question a small model can answer

A useful learning task might predict whether a unit should retreat from features such as normalized health, nearby threat and available cover. Define the target, label-generation procedure and evaluation conditions before training. A model that predicts a rule-generated label may simply approximate that rule; success does not demonstrate discovery of superior strategy. Keep the rule itself as a baseline.

Training changes parameters using examples. Inference applies frozen parameters to new inputs. A game must budget inference time, model memory, feature construction and fallback behavior. Training speed is not the runtime metric. The core exercise uses a small linear logistic classifier implemented with the standard library so the arithmetic is visible and no GPU is required.

Derive one gradient step

For features x and parameters w,b, compute z=w dot x+b and p=1/(1+exp(-z)). For binary label y, cross-entropy loss is -y*log(p)-(1-y)*log(1-p). Combining the derivative of the sigmoid with the derivative of this loss gives dL/dz=p-y. Therefore dL/dw=(p-y)*x and dL/db=p-y. This simplification is an application of the chain rule. Automatic differentiation generalizes that bookkeeping through larger computation graphs [S11].

Use a numerically stable sigmoid branch for large positive and negative z, and a stable loss expression rather than taking log(0). Reject non-finite features. Standardize or bound features using statistics fit only on training data. A confidence-like score from a classifier is not automatically a calibrated probability under a new game distribution.

Worked update

Let x=(1,2), y=1, initial w=(0,0), b=0 and learning rate 0.1. Then z=0 and p=0.5. The weight gradient is (-0.5,-1) and the bias gradient is -0.5. One gradient-descent step gives w=(0.05,0.10) and b=0.05. The new logit is 0.30 and probability about 0.5744. This is a one-example update, not evidence that the model generalizes.

Finite-difference checks on a tiny parameter vector can verify gradients. Compare the analytic derivative to [L(w+epsilon)-L(w-epsilon)]/(2*epsilon), using a sensible epsilon and a toleranced comparison. A small training loss does not validate a wrong gradient if the task is trivial or the implementation leaks labels.

Split by the source of correlation

Frames from one episode are correlated. Randomly splitting individual frames can place nearly identical situations from the same trajectory in both training and test sets. Split complete episodes, and sometimes maps, players or seeds, according to the intended generalization question. Keep a validation set for model selection and an untouched test set for the final estimate. Never tune a threshold repeatedly on the reported test set and still call it unseen.

Record episode IDs, data-generation seeds, feature schema, model parameters, optimizer settings and the split rule. Synthetic labels should be documented as synthetic. Avoid collecting personal player data when a synthetic task satisfies the educational goal. Report class balance and a confusion matrix so a model that always selects the majority action cannot hide behind accuracy alone.

Deployment quality includes failure behavior

Test an out-of-range health value, missing feature, unsupported model version, corrupted weights and an inference budget overrun. A safe fallback should still return a legal game action. For a tactical model, a rules-based conservative action may be appropriate. Model confidence alone should not override authoritative movement or reward validation.

Measure feature construction and inference separately. Run enough repeated calls to reduce timer granularity effects, but report batch size and distinguish batched throughput from one-agent latency. A toy Python measurement is not a claim about native engine performance. The full lab must compare a frozen model and the rule baseline on independent episodes, include an error analysis and measure the selected deployment path.

Responsible interpretation

A learned policy is not necessarily more explainable, fairer or more efficient than a rule system. Evaluate the purpose it actually serves. In the capstone, a simple authored AI remains acceptable; the learning lab is still required as a comparison exercise. The learner should be able to explain when training is worth its data and maintenance costs and when a transparent rule wins. A negative result is valid engineering evidence, provided the experiment and baseline are sound.

Original worked checkpoint

A model trained and tested on adjacent frames from the same recorded match may appear accurate by exploiting near-duplicates. Holding out complete matches provides a stronger test of generalization.

Full practical assignment

Train a small classifier or value estimator on synthetic game data and compare it with a rules baseline. Pass: independent splits, error analysis, inference timing, reproducible seeds and a fallback behavior are reported.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **One stable logistic gradient step**. Run `starter/python/demos/lesson_28_4.py`; implement `starter/python/exercises/lesson_28_4.py`.

Assigned study and native transfer

R46 : Machine Learning and Reinforcement Learning; R47 : linear models, multilayer perceptrons, backpropagation and generalization.

Read S11 on computation graphs and automatic differentiation. Use the original probability/statistics material to review evaluation and uncertainty. No PyTorch dependency is required for the core kernel.

Engine transfer: Export a versioned model and feature schema, then integrate a bounded inference adapter with a rules fallback. Keep model output advisory; authoritative simulation still validates legal actions.

Check your understanding

1. Why separate complete episodes across data splits?
2. What mathematical rule underlies backpropagation?
3. Why retain a rules-based baseline?
4. For x=(1,2), y=1, w=(0,0), b=0 and learning rate 0.1, compute one logistic cross-entropy gradient-descent update.
5. A classifier scores 98% when frames are randomly split, but 72% when entire episodes are held out. Which estimate better supports new-episode use, and why?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 28 project

Build an Inspectable Courier Agent. The agent travels to a target, responds to a blocked route and chooses among a small set of understandable actions. Give every decision a readable trace, including invalid alternatives and action cancellation. Use a strict search/replan budget and a safe behavior when it is exhausted.

Compare a rules baseline with one search-based or learned component. The lesson on learning still requires its synthetic training/evaluation exercise even when the integrated agent uses search. Select evaluation scenarios before looking at results: a clear route, a narrow corridor, a newly blocked passage, a missing target and at least one adversarial case.

Report completion/failure categories, time or work per decision, cancellation/resource behavior and reproducible seeds. For learned behavior, use independent episode splits and a frozen artifact. For search, provide legal-move/route checks and a completed-iteration fallback. A baseline outperforming the sophisticated component is useful engineering

evidence.

Deliver a behavior specification, scenario manifest, raw traces, oracle comparisons, cost distributions and a short design decision explaining the chosen approach. A reviewer should cause a failure and explain the recovery from the trace without reading every source line.

Module 28 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. A* found a route, but the agent never reaches the target. Name four different layers to inspect.
2. How do hysteresis and action cancellation solve different AI problems?
3. A search expires before completing the first root move. What can it safely return?
4. A classifier scores 99% after randomly splitting frames from the same matches. Why is that evidence weak?
5. How should a learned component behave when the model fails to load or inference is too slow?

Module 29: Multiplayer simulation and online services

Prerequisites: Modules 18-20 and 24; Module 26 for physical interaction.

Original reading IDs: R17, R20, R26. See the reference directory.

Module goal: Create a local two-client multiplayer demonstration with a network simulator for delay, jitter, loss, duplication and reordering. No public server or paid cloud account is required. Document authority and recovery rules.

Lesson 29.1: Authority, prediction and reconciliation

Learning goal: Provide responsive local control without trusting arbitrary client outcomes.

Decide who owns truth

An authoritative server accepts player intentions and computes permitted state transitions. A client can request movement; it cannot simply declare a new position, reward total or health value and expect trust. Authority is a game-design and security boundary. The local lab uses synthetic clients and no public service, but it must still validate inputs and ownership.

Define a message schema with protocol version, session/epoch, player identity, input sequence and bounded control values. The server chooses the simulation timestep. A client-supplied dt can otherwise become a speed multiplier. Authentication establishes a principal; authorization decides which actor that principal may control. This lesson's local harness models those roles without claiming to implement production identity or encryption.

Prediction hides response delay

A client immediately simulates its own legal input using the same movement rule as the authority. It retains unacknowledged inputs. The server later reports an authoritative state and the highest contiguous input sequence included in that state. The client restores that state, removes acknowledged inputs and replays the remaining ones in sequence. This is reconciliation. Epic's networked-movement documentation is a concrete engine comparison [S12].

The acknowledgment must identify exactly which input effects the state contains. The largest sequence number merely received is not enough when earlier messages are missing. Duplicate inputs must not be applied twice. A server can buffer out-of-order commands within a bound or use an explicit late-input policy. It must not leave an unbounded queue waiting forever for a missing packet.

Worked acknowledgment trace

The server sends x=10 after processing input 40. The client has predicted inputs 41,42,43,44, each moving +1 under this teaching rule. Reconciliation sets predicted x to 10 and replays those four inputs, yielding 14. Replaying input 40 again would move to 15 and double-apply acknowledged work. Snapping to 10 without replay would discard valid pending input and feel unresponsive.

Now suppose input 43 attempts an illegal +50 move while legal inputs are -1,0,+1. The server rejects or substitutes a neutral input according to its contract. The client may have predicted incorrectly, so a correction is expected. Validation must be performed before the authoritative update; merely clamping the final rendered position does not repair the server's trusted state.

Logical correction versus visual smoothing

The authoritative correction must affect future simulation immediately at a known tick. Visual smoothing can gradually reduce a render-only offset so the player does not see a harsh jump. Do not smooth the logical collision state toward authority while continuing to predict from the wrong state; that can create persistent divergence. Keep the simulation position and presentation offset separate.

Prediction becomes harder with interacting physics, nondeterministic code and shared objects. A simple movement example does not establish full networked rigid-body correctness. Start with one-dimensional or grid movement, then state which collisions and interactions are predicted and which wait for authority. A mismatch between client and server movement code is a correctness bug, not ordinary network jitter.

Transport faults and recovery

Build a seeded delivery queue that can delay, drop, duplicate and reorder messages. Keep a bounded command history on the client. If an acknowledgment is older than the last applied authoritative snapshot, ignore it under the protocol's sequence comparison rule. The teaching code uses unbounded Python integers for sequence numbers; a production fixed-width protocol requires an explicit wraparound comparison window.

Persistent packet loss cannot be defeated by finite retries with a guaranteed delivery claim. Define a retry budget and a resynchronization or disconnect state. To test convergence, stop generating new inputs and enter a declared reliable drain phase. Then both sides should agree on all accepted input effects. Label that phase rather than implying arbitrary permanent loss always converges.

Deliberate practice

The kernel reconciles an acknowledged state with a sorted, validated pending input list. The full lab runs two distinct clients and an authority through fault injection, records corrections and validates ownership. Include duplicates, a missing middle sequence, stale snapshots, invalid movement and reconnect with a new epoch. Report the final accepted input set and state, not only an animation of two dots.

In the engine, input capture produces commands; simulation consumes commands; networking transports them; presentation displays predicted or interpolated snapshots. Keep those layers independent so a local replay can reproduce a network bug without a live server. A paid cloud account is unnecessary for this learning objective.

Original worked checkpoint

The server acknowledges input 40. The client corrects to the returned state for that point and replays locally stored inputs 41 through 44, rather than applying those inputs twice to the wrong starting state.

Full practical assignment

Implement a one-dimensional or grid movement server with client prediction. Pass: invalid movement is rejected and induced delay produces documented corrections without permanently diverging state.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Input acknowledgment and reconciliation**. Run `starter/python/demos/lesson_29_1.py`; implement `starter/python/exercises/lesson_29_1.py`.

Assigned study and native transfer

R17 : transport semantics; R26 : networking and multiplayer concepts; R20 : networking in games.

Read S12 on client prediction, saved moves and correction. Compare the concepts rather than assuming the teaching protocol reproduces Unreal internals. Revisit Modules 18-20 for concurrency and trust boundaries.

Engine transfer: Create an engine command schema and separate authoritative, predicted and render-only state. Record one correction with the exact acknowledged and replayed input IDs.

Check your understanding

1. What is authoritative state?
2. Which inputs are replayed after acknowledgment of input 40 in the example?
3. Why send intentions rather than unrestricted reward totals?
4. Authority acknowledges input 40 at x=10. Pending inputs 41 through 44 each move +1. What is the reconciled prediction?
5. Why is acknowledging the highest received sequence unsafe when an earlier input is missing?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 29.2: Snapshots, interpolation, relevance and lag compensation

Learning goal: Manage bandwidth and latency while making fairness assumptions explicit.

Remote actors are usually presented in the past

A local actor can be predicted from its inputs. A remote actor's future intentions are unknown. Snapshot interpolation buffers recent authoritative states and renders a time slightly behind the newest known server time. This trades extra presentation delay for smoother behavior under jitter. The original networking article by Glenn Fiedler is the assigned conceptual comparison [S13].

Use server ticks or a mapped server-time estimate, not packet arrival time, as the interpolation axis. Arrival time includes variable transport delay. Let snapshots bracket render time t_r at t0 and t1. With t0<t1, alpha=(t_r-t0)/(t1-t0), and a linearly interpolated position is (1-alpha)*x0*+*alpha*x1. Orientation needs the rotation interpolation rules from animation.

Worked timeline

Snapshots at server times 1.00 s and 1.10 s contain x=2 m and x=4 m. Rendering at 1.04 s gives alpha=0.4 and x=2.8 m. If the newest estimated server time is 1.14 s, a 0.10 s interpolation delay leads to that 1.04 s render time. The transport may have delivered both snapshots almost simultaneously; their arrival spacing must not replace their simulation spacing.

When no later snapshot exists, choose a bounded extrapolation or hold policy. Extrapolation can improve brief gaps but may move an actor through a wall or around a turn it never took. A teleport or respawn is a discontinuity and should reset or segment the interpolation history. Never interpolate across different entity generations or match epochs.

Ordering and bounded history

Receive snapshots into a timestamped, deduplicated structure. A late packet may still be useful if it fills the current interpolation bracket; a packet older than the retained window can be discarded. Decide whether your simpler implementation retains out-of-order samples or accepts only increasing snapshots. Both require explicit consequences and tests. Bound memory, timestamps and entity counts before accepting untrusted data.

The kernel sorts unique timestamp/position samples, rejects conflicting duplicates and holds endpoints outside the available interval. The full lab must add a missing-snapshot policy, an interpolation-delay setting and a graph or table of perceived delay versus correction/jitter behavior. A smooth result can still have unacceptable latency, so measure both.

Delta compression requires a known baseline

A delta snapshot describes changes relative to a particular baseline ID. The receiver must possess that exact baseline. Guessing the most recent available snapshot can reconstruct plausible but wrong state. If the baseline is missing, request or await a full state according to the recovery policy. Retain baselines only within a bounded acknowledgment window. Compression is an optimization layered on a correct state protocol.

Calculate bandwidth from actual serialized bytes, snapshot rate and recipients. For example, 40 bytes per entity, 100 relevant entities and 20 snapshots/s produce 80,000 bytes/s of payload per client before headers and retransmission. Do not describe that as the total server bandwidth without multiplying by clients and including protocol overhead.

Relevance is also an information boundary

Do not send hidden enemy state merely because the normal renderer chooses not to display it. A modified client can inspect received data. Determine relevance on the authority using a visibility or gameplay policy, then serialize only allowed fields. Relevance and compression solve different problems: one selects what may be known; the other encodes it efficiently. A spectator may have different permissions from a player.

Lag compensation re-evaluates an action against a bounded historical world to account for delay under a game-specific fairness policy. A client's claimed timestamp is not unlimited permission to rewind. Bound the rewind horizon, validate timing plausibility and document whose experience is favored. The core lab need not implement a full shooter hit-registration system, but it must explain this tradeoff accurately.

Deliberate practice

Add a second remote actor to the network harness, interpolate its snapshots and filter a hidden entity from one client. Capture serialized output to prove exclusion before rendering. Inject out-of-order snapshots, a missing baseline and a teleport. Report buffer delay, held/extrapolated frames, dropped history and full-state recovery. In your engine, keep remote presentation state separate from the collision authority and from local prediction.

Original worked checkpoint

Buffering two snapshots 50 ms apart can interpolate motion during a delay variation, but it intentionally displays a slightly older world. This is a latency-quality tradeoff, not free smoothing.

Full practical assignment

Add remote interpolation and relevance filtering. Pass: out-of-order snapshots are handled, missing baselines trigger recovery, and the report explains both visual delay and hidden-state exposure policy.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Timestamped snapshot interpolation**. Run `starter/python/demos/lesson_29_2.py`; implement `starter/python/exercises/lesson_29_2.py`.

Assigned study and native transfer

R20 : networked game state; R26 : replication and network smoothing; revisit Module 19 message validation.

Read S13 on interpolation buffers and linear interpolation. Revisit S12 for an engine comparison. Derive the payload bandwidth of your actual schema.

Engine transfer: Implement authority-side relevance filtering and capture the outgoing payload. Demonstrate that a hidden entity is absent from the unauthorized client, not merely invisible in the renderer.

Check your understanding

1. What cost does snapshot interpolation commonly add?
2. Why must delta-compression baselines be identified?
3. Why avoid transmitting irrelevant hidden enemy state?
4. Positions are 2 m at 1.00 s and 4 m at 1.10 s. What is the interpolated position at 1.04 s?
5. A delta names baseline 120, but the receiver only retains baseline 121. Should it apply the delta to 121?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 29.3: Lockstep, rollback and deterministic replay

Learning goal: Choose synchronization around the game model rather than genre slogans.

Input lockstep and rollback make different latency choices

Lockstep advances a tick only after the required input bundle is known. When all participants run the same deterministic transition, exchanging inputs can be cheaper than exchanging full state. Waiting for delayed input can stall play. Rollback predicts missing input, saves earlier states, and re-simulates when the actual input arrives. GGPO is an original rollback SDK reference, not a claim that this package integrates it [S14].

Define the state boundary before implementing rollback. Let S_t be the state before tick t and I_t the input for that tick. Save S_t, predict or receive I_t, then compute S_(t+1). If input for tick k changes, restore S_k and re-run ticks k through the latest completed tick. Restoring S_(k+1) instead is an off-by-one error that preserves the effect of the wrong input.

What must be saved

A snapshot needs every future-affecting deterministic field: positions, velocities, gameplay timers, entity generations, random state, spawn counters, AI state and necessary solver caches. External pointers are not portable state. A copied array of visible transforms is usually insufficient. Version the schema and test the next tick after restore, not just the restored values.

At 60 Hz, 120 saved ticks cover two seconds. With 2 MB per state, raw snapshots alone consume 240 MB under decimal MB units. Metadata, input history and event records add more. Compression or incremental state can reduce storage, but also adds restore cost and failure modes. Measure the complete history budget rather than quoting the smallest structure in isolation.

Worked replay correction

A scalar simulation starts S_0=0 and consumes inputs [1,1,0,1]. Predicted post-step states are 1,2,2,3. Later, the actual input at tick 2 is -1 rather than 0. Restore S_2=2, apply -1 to obtain S_3=1, then apply tick 3's +1 to obtain S_4=2. The corrected final state is 2. Replaying all inputs from a known initial state is a useful independent oracle for the small test.

Now add an event when the state first reaches a target. Re-simulation may regenerate the same event. Presentation deduplicates by stable identity; durable rewards wait for an appropriate confirmation boundary. Sending a payment, file write or external message directly from a replayed tick creates repeated irreversible side effects. Separate deterministic event generation from external commitment.

Determinism is still required within the chosen scope

Fixed dt alone does not guarantee deterministic replay. Unstable pair ordering, unseeded randomness and excluded caches can diverge. Use the first-divergence harness from Lesson 26.4. Deliberately change one operation or input and verify that the report identifies the earliest differing state. Glenn Fiedler's lockstep article is a useful description of the assumptions and buffering tradeoffs [S24].

A late input older than retained history cannot be repaired by restoring a state that no longer exists. Define a recovery policy: request full authoritative state, pause for resynchronization, or disconnect under a clear error. Never silently apply the old input to the current tick. Bound the maximum rollback depth and re-simulation work per frame. A large correction can exceed the frame budget even when each ordinary tick is fast.

Test side effects, not just positions

Run a delay scenario that forces repeated replay across the same event tick. Count deterministic event generations, unique presentation events and committed durable effects separately. It is normal for deterministic generation to occur several times during replay; it is not normal for a reward to be committed several times. Keep the deduplication cache outside the restored simulation snapshot.

Use a stable match epoch so a new match can legitimately generate an event with the same tick and entity number. Retain only the event history needed for the rollback window, plus any durable idempotency records whose lifetime belongs to a different system.

Deliberate practice

The kernel replays an integer input tape after a correction and returns the corrected state sequence. The full lab adds a bounded snapshot ring, delayed input, first-divergence hashes and side-effect suppression. Compare a corrected run to a run that knew all inputs from the start. Include the oldest retained tick, an input one tick too old, duplicate late input and a changed random seed. Submit a timing and memory report for worst observed re-simulation depth on the declared environment.

Original worked checkpoint

At 60 simulation ticks per second, storing 120 snapshots covers two seconds of history. If each snapshot is 2 MB, raw snapshot storage alone reaches 240 MB before metadata or compression.

Full practical assignment

Implement rollback or input lockstep for the tiny deterministic game. Pass: a delayed-input test converges, state hashes detect a deliberately introduced divergence, and audio or reward side effects are not duplicated.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Rollback to the pre-tick state**. Run `starter/python/demos/lesson_29_3.py`; implement `starter/python/exercises/lesson_29_3.py`.

Assigned study and native transfer

R20 and R26 : multiplayer concepts; derive the implementation using Modules 18, 24 and 26 rather than assuming engine-specific guarantees.

Read the S14 SDK overview and S24 on deterministic lockstep. Reuse the serialization and first-divergence evidence from Lesson 26.4; do not begin from a transform-only snapshot.

Engine transfer: Add a bounded rollback ring to your engine or a separate deterministic two-client demonstration. Keep irreversible side effects behind an explicit confirmation/idempotency boundary.

Check your understanding

1. How much time do 120 ticks cover at 60 Hz?
2. How much raw storage do 120 snapshots of 2 MB require?
3. Why are external side effects difficult during rollback?
4. With S_0=0 and predicted inputs [1,1,0,1], tick 2 is corrected to -1. Which state must be restored and what is the corrected S_4?
5. A late input refers to a tick older than the snapshot ring. What is a valid response, and what is an invalid shortcut?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 29.4: Match services, identity, persistence and failure recovery

Learning goal: Design a small online service without treating it as part of the render loop.

Online services are state machines with failures

A match registry might move through created, joining, active, completing and closed states. Write allowed transitions and which principal may trigger each. Authentication says who is making a request; authorization says whether that principal may act on this match. A client-supplied player name is not authentication. The local lab uses synthetic principals provided by a trusted test harness and is not a production identity service.

A service request can fail before processing, after committing but before the reply, or while the caller times out without knowing the outcome. Retrying is necessary, but retrying a reward operation naively can duplicate its effect. The important question is not whether the network delivered a message exactly once; it is whether the durable effect occurs once under the defined retry contract.

Idempotency needs more than a set in memory

Assign a stable operation ID, such as a match-result ID, and bind it to the canonical payload. On first valid submission, record the operation and apply its effect in one database transaction. On an exact retry, return the prior outcome without applying the effect again. If the same ID arrives with a different payload, reject the conflict rather than silently accepting a different result.

The operation record and the reward update must commit together. Recording the ID first and then crashing before the reward can permanently lose the effect. Applying the reward first and crashing before recording the ID can duplicate it on retry. SQLite's transaction and atomic-commit documentation explains the durable transaction model used for the local comparison [S15, S16]. Actual durability also depends on configuration, storage and failure assumptions.

Worked timeout after commit

The authority submits result R17 awarding 50 points to player P. The store starts a transaction, inserts R17 with its payload, increments P's points and commits. The response is lost. The authority retries R17. The store finds the same payload and returns 'already applied' with no additional points. P has 50, not 100. A retry of R17 awarding 500 points is a conflict and must fail.

Now restart the store process between submissions. An in-memory set forgets R17; a durable operation table does not. The lesson therefore requires a file-backed local database test, not only a dictionary. Parameterize SQL inputs and validate numeric ranges. Do not construct a query by concatenating an untrusted name or operation ID.

Authorization before mutation

Only a trusted match authority should commit its result. A player may read permitted match information but must not edit another player's reward. Check that the submitting principal owns the active authority lease or meets the lab's simpler explicit role rule. Re-check necessary state within the transaction to avoid a stale authorization decision racing with a state change.

For a production system, tokens, key rotation, transport security, abuse controls and account recovery would be additional work. They are intentionally outside this bounded local lab. Do not expose its synthetic identity switch on a public network or describe it as secure authentication.

Bounded retry and recovery

Use a finite retry budget with backoff and a user-visible pending or unavailable state. Retry only operations that are safe under the protocol. A permanent validation error is not repaired by retrying forever. Distinguish timeout, authorization failure, conflict and unavailable storage in both code and user feedback.

A lease or heartbeat can help detect an abandoned match, but absence of a heartbeat is uncertainty rather than proof that the other process has stopped. Avoid two simultaneous authorities by defining lease ownership and version checks. The core assignment can use a single local authority and still document what would need to change for multiple processes.

Deliberate practice

The kernel validates principal, payload and idempotency against an in-memory model. The integration companion uses SQLite to test commit, exact retry, conflict, unauthorized access and restart persistence. The full lab adds a match-registry state machine and service-loss behavior. Inject a failure immediately before commit and immediately after commit, then retry with the same operation ID.

Submit a state-transition table, operation schema, test logs and a client outcome for each failure class. Keep saves and local player progress separate from temporary match state. No paid cloud, real accounts, payment processing or public deployment is required. The engineering objective is a coherent failure-aware contract, not a large online platform.

Original worked checkpoint

A match-result submission is retried after a timeout. A stable operation identifier lets the progression service recognize the duplicate and avoid granting the same reward twice.

Full practical assignment

Build a local match registry and idempotent result store. Pass: duplicate submissions do not duplicate rewards, unauthorized edits fail, and simulated service loss produces a documented client outcome.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Idempotent result-store model**. Run `starter/python/demos/lesson_29_4.py`; implement `starter/python/exercises/lesson_29_4.py`.

Assigned study and native transfer

R18 from the directory: transactions; R17 : transport; revisit Modules 19-20 for distributed failure and threat modeling.

Read S15 on transactions and S16 on atomic commit. Review Module 20 trust boundaries. Treat the supplied synthetic-principal code as a local test model, not production authentication.

Engine transfer: Implement a file-backed local result store and prove exact retry behavior across a process restart. Add a match-state transition table and an unavailable-service client state.

Check your understanding

1. Is authentication the same as authorization?
2. Why identify a match-result operation uniquely?
3. Must this lab use a paid public cloud?
4. A result commits, but its response is lost. The client retries with the same operation ID and payload. What must happen to the reward total?
5. Why must a reused operation ID with a different payload be rejected instead of treated as a normal retry?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 29 project

Build a Local Two-Client Simulation. Use one authoritative process and two local clients, or an equivalent process/socket layout with explicit identities and controlled inputs. The package’s in-process simulator is a reference for message semantics, not a substitute for the student’s transport adapter. No public endpoint or paid service is required.

Exercise delay, jitter, loss, duplication and reordering with seeded schedules. Include local prediction/reconciliation, a remote interpolation buffer and one missing-baseline recovery. Show which state is omitted by relevance filtering. Use rollback or input lockstep for a tiny deterministic mode, with a documented history/barrier policy and side-effect handling.

Add a local registry and transactional result store. Inject failure before and after commit and restart the service. Keep synthetic test identities clearly distinct from production authentication. Demonstrate an explicit client state while the service is unavailable rather than silently losing results.

Deliver protocol schemas, process topology, fault schedules, correction/replay traces, hidden-state payload audit, transaction/restart logs and limitations. Review asks what is authoritative, what is predicted, what can be replayed and what must be idempotent.

Module 29 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. The server receives inputs 10, 12 and 13 but not 11. What may it acknowledge under the contiguous-ACK protocol?
2. Is hiding an entity in the renderer sufficient to prevent clients from learning its position?
3. Explain the off-by-one error in restoring S_(t+1) when replacing input I_t.
4. A reward committed but the response was lost. How does the client retry without duplicating the reward?
5. What does an in-process faulty-network test prove, and what remains to test?

Module 30: Production quality and the undergraduate engine capstone

Prerequisites: Modules 21-29 plus Gate D software-engineering evidence.

Original reading IDs: R20, R26, R29, R45. See the reference directory.

Module goal: Gate F: release a small original game using your engine, with one complete gameplay loop, settings, save/load, diagnostics, accessibility checks, a tested distribution and a technical postmortem. Scope is judged by quality of evidence, not visual size.

Lesson 30.1: Profiling, frame budgets and performance regression tests

Learning goal: Optimize measured bottlenecks without sacrificing correctness.

Profile the complete experience

A game can average 60 frames per second and still stutter. Store raw frame durations, not only an average FPS counter. Describe what each timer measures: CPU simulation, render submission, GPU execution, present wait or end-to-end frame interval. CPU and GPU work may overlap, so summing every timer can double-count elapsed time. Preserve the timing boundaries established in Volume 5.

Choose a fixed benchmark scene, input tape, resolution, build configuration and warm-up policy before optimizing. Record CPU/GPU model where applicable, operating system, compiler, flags, power conditions and background load. A software-renderer path is valid when clearly labeled. Do not claim GPU timing from a CPU timer around an asynchronous submission.

Distributions and measurement boundaries

A nearest-rank percentile sorts N observations and selects index ceil(p*N)-1 for p in (0,1]. Different statistical packages may use interpolation instead, so record the convention. For durations [10,10,10,10,50] ms, nearest-rank p95 is 50 ms and the mean is 18 ms. A tiny sample is useful for checking code but weak evidence about a production tail distribution.

With 1,000 samples, nearest-rank p99 uses rank 990. Keep the raw series so a reviewer can distinguish isolated stalls, periodic spikes and warm-up effects. Report multiple independent runs and the number of samples, not a single impressive minimum. Timing noise is not fixed by printing more decimal places.

Amdahl's law as a scope check

If a 20 ms frame spends 2 ms in a subsystem, making that subsystem ten times faster reduces it to 0.2 ms. The unchanged 18 ms remains, so the ideal new frame is 18.2 ms, a speedup of about 1.099. Even an infinitely fast replacement cannot reduce this modeled frame below 18 ms. This arithmetic assumes the measured work is on the relevant critical path and that optimization does not change other costs.

A microbenchmark can improve while the frame gets slower because of extra allocations, cache pressure, synchronization or data conversion elsewhere. Measure before and after in the same workload and verify correctness. A faster collision routine that drops contacts is not an optimization. A faster model that changes decision quality needs a quality/performance comparison, not a pure speed claim.

Regression thresholds need context

A useful performance test has a declared workload and an acceptance rule. For a stable dedicated machine, a percentile threshold may be meaningful. On shared CI, use broader thresholds, repeated runs and a review path for noisy failures. Keep functional tests separate so performance noise does not obscure correctness regressions. A budget threshold is a project target, not a universal statement about what every machine should achieve.

Track work counts alongside time: bodies, contacts, active voices, navigation expansions, rollback ticks, draw calls and bytes loaded. A frame spike after adding 100 enemies may be expected scaling rather than a compiler regression. Normalize only when the normalization answers the question; hiding a doubled workload behind 'time per entity' can conceal a worse player experience.

Worked optimization experiment

Suppose a deterministic scene spends most simulation time rebuilding an unchanged navigation grid. Hypothesis: versioning the static grid will remove repeated work. First retain the original implementation as a correctness oracle. Run the same input tape through both and compare paths and gameplay state. Then collect several before/after frame series with identical warm-up and configuration. Record any added memory for cached data and how invalidation works when a door changes.

If the result improves median time but increases rare spikes due to cache rebuilds, report both. Consider whether rebuilding incrementally or at a controlled loading boundary better fits the game. The best result is a measured design tradeoff, not the largest isolated speedup percentage.

Deliberate practice

The kernel calculates finite, positive frame-time statistics using the declared nearest-rank rule. The full lab profiles the capstone, optimizes one verified bottleneck and stores before/after raw data. Include a correctness comparison, repeated runs, peak-memory measurement and a limits statement. Synthetic demo timings are labeled synthetic; locally measured test-harness timings are not presented as capstone performance.

Build a regression test that another person can run from the package. Explain one change that failed to improve performance and why the evidence changed your plan. A negative optimization result can demonstrate stronger engineering judgment than an unsupported claim of a dramatic speedup.

Original worked checkpoint

A subsystem taking 2 ms of a 20 ms frame can save at most 2 ms if removed entirely. A 10x speedup in that subsystem alone yields an 18.2 ms frame under the simplified additive model.

Full practical assignment

Profile the capstone scene, optimize one verified bottleneck and add a regression benchmark. Pass: repeated before/after results, unchanged correctness and disclosed measurement uncertainty are included.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Nearest-rank frame statistics**. Run `starter/python/demos/lesson_30_1.py`; implement `starter/python/exercises/lesson_30_1.py`.

Assigned study and native transfer

R26 : profiling and optimization; R45 : sanitizer diagnostics; revisit Modules 09 and 18.

Revisit R20 production and R26 profiling concepts; read S19 for the distinction between simulation catch-up and rendering cadence. Preserve Volume 5 CPU/GPU timing definitions.

Engine transfer: Add a reproducible capstone benchmark with raw frame durations, work counts and environment metadata. Keep measurement code out of the authoritative replay hash.

Check your understanding

1. What is the ideal frame time after a 10x speedup of the example 2 ms subsystem?
2. Why record build type?
3. Why can a microbenchmark improvement fail to help the game?
4. Using nearest-rank percentiles, what are the mean and p95 of [10,10,10,10,50] ms? Is this enough data to characterize a real game tail?
5. A 20 ms frame contains 2 ms of optimizable work. What is the ideal total time after a 10x improvement, and what is the infinite-speed lower bound?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 30.2: Compatibility, accessibility, localization and human testing

Learning goal: Test whether people can actually use the complete product.

Test supported experiences, not a vague platform list

A compatibility matrix states exactly what was tested: operating system and version, display size, input device, audio availability, renderer/backend and build configuration. 'Works on PC' is not a test result. Mark untested combinations as untested rather than inferred. A minimum supported configuration is a project decision backed by measurements and task tests.

Prioritize the full gameplay loop: start, understand controls, perform the objective, recover from failure, save where supported, load and exit. Test a clean folder without development tools or hidden asset paths. Missing controller or audio devices should produce a usable fallback rather than a crash. Resizing, losing focus and exiting while loading are state transitions with ownership implications.

Input is an action map

Bind physical controls to semantic actions such as move_left or interact. The UI asks the binding system for the current display label; it does not hard-code 'Press E.' Rebinding must update gameplay, prompts, menus and saved settings consistently. Define conflict handling and a way to restore defaults. A user must not be able to bind away every route to the settings screen without a recovery path.

Avoid requiring simultaneous or held inputs when an alternate mode can preserve the intended experience. Test keyboard-only menu navigation and focus visibility. Microsoft XAG 107 provides the assigned input-design reference [S25]. A checklist does not establish that every player can use the game; observe the chosen tasks with appropriate participants.

Localization begins before translation

Store text as keyed resources with placeholders, not as scattered string literals. Format numbers, dates and plurals through an appropriate locale-aware layer where needed. Pseudo-localization expands or alters strings to expose clipping, concatenation assumptions and hard-coded text. Preserve placeholders such as {player} while expanding surrounding words.

A simple teaching pseudo-localizer can bracket text and add expansion characters. It is not a translator, font-shaping engine or full internationalization library. Test long labels, multiline text, missing keys and right-to-left layout when included in the support scope. Unicode code points, grapheme clusters and display width are not interchangeable. The full lab should use the engine's real layout system rather than predicting fit from character count alone.

Worked binding and text test

The default interaction key is E. Rebind it to Space and reopen the objective screen. The prompt must now identify Space, including after save/reload. Expand 'Activate relay' to a longer test string and reduce the window size. The control must remain readable and operable, perhaps through wrapping or layout growth. Shrinking text until it technically fits can violate the intended readability target.

Mute audio and repeat the relay task without coaching. A visual status change and readable message should carry the critical cue. Remove color as the sole distinction between inactive and active states. Microsoft XAG 101 addresses text display, and XAG 103 addresses additional information channels [S26, S23]. Record specific observations rather than claiming broad accessibility certification.

Human testing as evidence

Use voluntary participation, explain the task and collect only needed observations. A small informal test is useful for finding defects but is not a representative population study. Ask the tester to complete a task without step-by-step coaching, then record where they hesitated, failed or misunderstood. Do not treat polite enjoyment feedback as proof that controls are discoverable.

Write each issue with reproduction steps, affected configuration, severity, evidence and a retest result. 'Player got confused' is too vague; 'after rebinding interact, the objective prompt still showed E, causing three failed attempts' points to a fix. Do not store unnecessary identifying information in the report.

Deliberate practice

The kernel pseudo-localizes literal text while preserving brace placeholders and validates a small action-binding map. The full lab runs at two window sizes, with remapped controls, muted audio, missing devices and expanded text. Include at least one independent task test and retest a discovered issue. Mark user-testing evidence as pending until a real tester participates.

For the capstone, integrate settings early rather than adding them after every UI string and input check has been hard-coded. Readability, recovery and control customization are part of a finished gameplay loop. They are not cosmetic polish that can be waived because the physics tests pass.

Original worked checkpoint

A button sized for a short English label can clip a longer translated label. A control prompt that hard-codes a keyboard letter can become wrong after rebinding.

Full practical assignment

Test the game at two window sizes, with remapped input and an expanded pseudo-localized text set. Pass: no critical clipped controls, a muted-audio task remains solvable and observed usability issues are tracked.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Pseudo-localization and current binding prompt**. Run `starter/python/demos/lesson_30_2.py`; implement `starter/python/exercises/lesson_30_2.py`.

Assigned study and native transfer

R29 : input, text, audio, contrast and UI guidelines; R20 : playtesting and production.

Read S25 input, S26 text display and S23 additional channels. Read S07 for the limits of guideline-based review. Use the original R20 playtesting material for task observation.

Engine transfer: Centralize action bindings and text resources in the engine. Have an independent tester complete the loop with a changed binding and muted audio; record and retest actual issues.

Check your understanding

1. Why use pseudo-localization before full translation?
2. What should a control prompt show after rebinding?
3. Does checking every accessibility guideline prove legal compliance?
4. After interact is rebound from E to Space, which systems must reflect the change?
5. Why is an expanded pseudo-localized label useful but insufficient to prove support for all languages?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 30.3: Packaging, patching, modding and supply-chain boundaries

Learning goal: Distribute a game without creating an uncontrolled execution path.

A release is a reproducible boundary

The release folder must contain the executable or supported runtime entry point, required assets, defaults, notices, dependency information and launch instructions. It must not depend on an absolute path in the developer's home directory. Keep user saves outside replaceable installation files. Test an extracted package in a new folder before declaring it runnable.

An archive checksum detects a change in bytes when compared to a trusted expected value. A checksum shipped alongside an untrusted archive does not by itself authenticate the publisher; an attacker could replace both. Distinguish integrity, authenticity and authorization. The capstone requires provenance and integrity records, not a custom cryptographic update system invented without review.

Validate data-only mods before allocation

A data-only mod can still consume excessive memory, trigger integer overflow in native code or exploit a parser defect. Define a small versioned schema with allowlisted fields and bounded lengths, counts and dimensions. Reject unknown executable hooks. Check multiplication before allocating width*height*channels bytes. In a fixed-width language, use checked arithmetic or compare against the maximum before multiplication.

The core schema permits a title and a bounded list of resource dimensions; it does not execute scripts or extract arbitrary files. For archives, reject absolute paths, parent traversal, device paths and unexpected links, and enforce uncompressed-size and file-count limits before extraction. Python's ZIP documentation explicitly warns about untrusted archives and extraction concerns [S18]. The included validator is a narrow teaching contract, not a general secure archive extractor.

Worked allocation check

A texture declaring width=100000, height=100000 and four bytes per pixel requests 40,000,000,000 bytes, approximately 40 GB in decimal units. Even without integer overflow, this exceeds a small game's resource budget. Check each dimension and the total resource budget before allocation. Multiple individually acceptable textures can still exceed a package-wide cap, so validate the sum as well.

A mod schema version of 99 is not silently treated as version 1. An unsupported version should fail with a useful error before altering the current game state. Parse into a temporary validated object, then install it at a safe boundary. A partially applied mod is harder to recover from than a rejected one.

Interrupted updates and saves

Use staged version directories rather than overwriting the only valid installation in place. Build a candidate version, verify its manifest, test its entry point where feasible, then switch a small active-version pointer. Retain the prior version until the new one is accepted. If staging fails, the active version stays unchanged. This teaching model does not claim a production updater across all filesystems.

For a save file, write a temporary file in the same directory, flush it as appropriate, then replace the destination. Atomic replacement provides a useful visibility property on supported same-filesystem operations; durability across power loss also depends on flushing, directory metadata, filesystem and operating-system behavior. Python's os.replace documentation describes the rename/replace contract, not a universal crash-durability guarantee [S17]. Keep a version and integrity check inside the save format and preserve a known-good copy when the design needs one.

Supply-chain and capability boundaries

Record library versions, source locations, licenses and whether a dependency is bundled or system-provided. Review build scripts before running them. Never automatically download and execute arbitrary community code as a convenience feature. Executable plugins run with the permissions of their process unless meaningful isolation says otherwise. Calling a component a sandbox is not evidence of a capability boundary.

The capstone's mod requirement is deliberately data-only. A script or native-plugin extension is advanced optional work requiring an explicit threat model, resource limits and a separate review. Do not expand a small finished game into an unbounded marketplace project.

Deliberate practice

The kernel validates a JSON-like mod object and calculates resource limits without allocating its declared payloads. The integration companion stages a candidate update and simulates interruption before activation. The full lab must package the actual game, test a clean-folder launch, reject malformed/oversized mods and preserve a valid version through the stated interruption tests. Include interrupted-save behavior and prove that replacing application files does not erase user progress.

Submit a release manifest, provenance list, schema, rejection corpus and recovery report. A green unit test for one validator does not prove arbitrary external content is safe; state exactly which inputs and capabilities are supported.

Original worked checkpoint

A malformed mod declares a texture with dimensions large enough to overflow a byte-count calculation. Validation must check multiplication and total resource limits before allocation, even though the file is nominally data.

Full practical assignment

Package the game and implement a data-only mod with a versioned schema. Pass: a clean-folder launch works, oversized or malformed mods are rejected and an interrupted-update simulation preserves a valid version.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Bounded data-only mod schema**. Run `starter/python/demos/lesson_30_3.py`; implement `starter/python/exercises/lesson_30_3.py`.

Assigned study and native transfer

R20 : production; R26 : packaging and plugins; R45 and Module 20: defensive memory and dependency practices.

Read S17 os.replace and S18 ZIP extraction warnings. Revisit Module 20 dependency and trust-boundary practices and original R45 defensive native-code material.

Engine transfer: Create a versioned data-only mod schema and an actual release folder. Test rejected input, interrupted staging and a save that remains intact when installation files are replaced.

Check your understanding

1. Can a data-only mod still be dangerous to robustness?
2. Why separate saves from replaceable installation files?
3. Does naming a system a sandbox establish isolation?
4. A texture declares 100000 by 100000 pixels at four bytes per pixel. What allocation would it imply, and when should it be rejected?
5. Why does a SHA-256 checksum stored beside a downloaded archive not by itself prove publisher authenticity?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 30.4: Capstone review, release and technical postmortem

Learning goal: Demonstrate ownership of a complete engineering result.

Finish one loop and own the result

Gate F is a small original game built on the learner's engine, not a collection of unrelated demonstrations. A loop has an entry state, a goal, meaningful player actions, success or failure, and a restart or continuation path. The scope can be a single arena, puzzle or short tactical scenario. Commercial art quality, revenue and a large world are not the assessment target.

Carry forward the Volume 5 engine ownership, rendering and resource evidence. Integrate collision/physics, animation or articulated motion, bounded audio, an inspectable AI, settings, save/load and a tested package. The multiplayer requirement may remain a separate local two-client demonstration with delay, loss and replay tests. Do not force a public online service into the game merely to satisfy the networking module.

Integration reveals contract mismatches

The physics system may use meters while the renderer uses pixels; animation may propose root motion while movement also applies velocity; audio may receive an event after an entity has been destroyed. A capstone architecture review should identify who owns each state, which thread or tick phase may mutate it, and how data crosses the boundary. An attractive diagram without a failure trace is not enough.

Freeze a small feature set before the final evidence run. Record deferred work explicitly. Adding a new renderer, account system or mod marketplace during stabilization undermines the goal. A well-tested small system is stronger evidence than a large unfinished system whose critical paths remain unexercised.

Evidence must be independently reproducible

Provide source, build commands, assets with provenance, a release package, raw performance data, test results and known limitations. A second person must be able to build or launch from a clean checkout or extracted source tree using documented prerequisites. Missing tools should produce a clear error rather than a silently skipped test that reports success.

The reviewer finishes the full gameplay loop, reproduces a named bug regression and observes a live change. A recording can supplement these checks but cannot replace independent execution. The learner must explain supplied reference code, library use and AI assistance. The submitted work should distinguish what was authored, adapted, generated and tested.

Worked release decision

Suppose the game completes its loop, but saving during scene loading can corrupt the only save. The average frame time meets the chosen budget and the art looks polished. This is still a release blocker because ordinary use can destroy progress. Fix the ownership/atomicity defect, or disable the unsafe save path with a clear supported behavior and retest. A high score in graphics cannot compensate for a critical data-loss path.

Now suppose one optional high-quality shadow mode misses the target budget on an untested GPU. Do not claim that configuration is supported. Restrict the support matrix to measured configurations and preserve a functional fallback. Honest scope restriction is different from concealing a defect in a claimed supported path.

Technical postmortem

Write the postmortem around decisions and evidence. State the initial hypothesis, what was implemented, what measurement or test challenged it, and what changed. Include at least one failed approach. For example, a spatial grid may have been slower than brute force in a tiny clustered level; retaining the simpler method can be the right decision. A narrative of uninterrupted success teaches little.

Report performance boundaries, memory limits, compatibility gaps, user-test findings and unresolved nonblocking defects. Link each substantial claim to an artifact in the evidence directory. Do not fill the report with fabricated tester quotes or timings. Pending human review stays pending until it occurs.

Oral review and live modification

Be ready to trace one input from device binding through authoritative simulation to render and audio output. Explain a collision normal sign, a rollback restore tick, a callback ownership rule and one rejected mod. Then make a bounded change, such as changing the relay activation threshold, adding a supported key binding or inserting a collision regression test. Predict which tests and documents must change before editing.

An oral review should test understanding, not surprise the learner with unrelated trivia. The capstone specification includes prompts and a scoring rubric. A failed explanation leads to targeted repair and another attempt; attendance and quiz recall alone do not establish mastery.

Completion and the next volume

Pass the mandatory release blockers, meet the rubric threshold, and archive the exact reviewed revision. Gate F closes the undergraduate software-engineering capstone. The next established volume is Computer Engineering and Digital Design, Modules 31-35, not another game-feature expansion. Keep the software workload and measurement discipline: they will provide concrete motivation for later digital-design and hardware/software co-design work.

Original worked checkpoint

A small game with reproducible builds, bounded memory, clear controls and an honest bug list is stronger evidence of engineering ownership than an unfinished large world with no tests.

Full practical assignment

Complete Gate F and the oral questions in the handbook. Pass: another tester finishes a full loop, the release is reproducible, critical defects are resolved or clearly blocked from release, and the postmortem cites measured evidence.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Evidence-gate rubric evaluator**. Run `starter/python/demos/lesson_30_4.py`; implement `starter/python/exercises/lesson_30_4.py`.

Assigned study and native transfer

R20 : course projects and critique; revisit the project rubric and professional-practice lessons.

Revisit R20 project critique and the program Gate F specification. Use the supplied rubric, evidence map and oral review. No external textbook or generated reference implementation substitutes for independent capstone work.

Engine transfer: Archive the reviewed engine/game revision with a clean-build record, an independently completed loop, one reproduced regression and a live-change review. Do not mark human review complete in advance.

Check your understanding

1. What proves completion beyond a recorded demo?
2. Why disclose library and AI assistance?
3. Why include a live modification in an oral review?
4. A game meets its frame budget but an ordinary interrupted save can corrupt the only progress file. Can a high overall rubric score override this?
5. What should a technical postmortem contain beyond a list of completed features?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 30 project

Complete Gate F: the undergraduate software capstone. Integrate one original playable loop with the engine built in Volume 5. This is the culminating use of the module projects, not an additional unrelated large game. Reuse well-understood systems and keep content deliberately small.

Follow CAPSTONE_SPEC for scope, milestone evidence, test matrix and review. The multiplayer demonstration may remain separate from the main game. The game itself still requires collision/physics, animation or articulated motion, bounded audio, inspectable AI, settings, save/load, diagnostics and a tested distribution. Lesson 30.3’s data-only mod lab remains required even when the released game does not expose mods.

Collect repeated performance measurements on a declared machine, complete the compatibility/accessibility tasks, test failure recovery and arrange an independent loop completion and live change. A software-renderer route is acceptable where carried forward from Volume 5; do not claim GPU validation without a tested GPU backend.

Deliver the release, source/build instructions, provenance, raw evidence, reviewer record and postmortem. Grade the quality of reasoning and evidence, not art budget, map size or commercial success.

Module 30 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. A release’s mean frame time improves but its p99 doubles. Is it necessarily better?
2. What is wrong with marking an untested platform as compatible because the code is portable?
3. Does a data-only mod eliminate security and reliability risk?
4. Why keep both a numerical rubric and mandatory release blockers?
5. What belongs in a technical postmortem beyond a feature list?

Reference Directory

S-prefixed entries are the focused primary-source companions consulted for this expansion, September 28, 2026. Numbers are stable identifiers; gaps are intentional. R-prefixed entries preserve the original program reading directory. External books, papers, documentation and library source are not bundled; the explanations, exercises and demonstration code in this package are original teaching material. Pin installed versions in your own evidence rather than treating a mutable documentation page as an installed build.

S01: Collision documentation

Box2D project. Shape queries, broad phase, dynamic tree and time-of-impact limitations.

Open primary source

S02: Simulation documentation

Box2D project. Stepping, contacts, solver settings and the documented determinism contract.

Open primary source

S03: glTF tutorial: Skins

Khronos Group. Joint transforms, inverse bind matrices and mesh-node correction.

Open primary source

S04: glTF 2.0 specification

Khronos Group. Normative asset, animation and skinning data contracts.

Open primary source

S05: Writing a callback function

PortAudio project. Callback constraints and real-time-safe execution discipline.

Open primary source

S06: SDL_AudioStreamCallback

SDL project. SDL3 audio stream callback contract; consult the API for the chosen integration.

Open primary source

S07: Xbox Accessibility Guidelines

Microsoft. Game accessibility design/testing guidance, not a legal-compliance certification.

Open primary source

S08: CS188: Informed search

University of California, Berkeley. A*, heuristics and search guarantees under stated assumptions.

Open primary source

S09: Introduction

BehaviorTree.CPP project. Behavior-tree concepts, statuses and implementation context.

Open primary source

S10: CS188: Minimax

University of California, Berkeley. Adversarial search and alpha-beta reasoning.

Open primary source

S11: Automatic differentiation with torch.autograd

PyTorch project. Gradient/parameter concepts; framework use is optional for the core synthetic exercise.

Open primary source

S12: Networked movement in the Character Movement Component

Epic Games. A maintained engine comparison for authority, prediction and correction.

Open primary source

S13: Snapshot Interpolation

Glenn Fiedler. Author’s technical explanation of interpolation, jitter and visual delay.

Open primary source

S14: GGPO SDK repository

GGPO authors. Primary implementation/documentation for rollback networking concepts.

Open primary source

S15: Transaction documentation

SQLite project. Transaction boundaries and behavior.

Open primary source

S16: Atomic Commit in SQLite

SQLite project. Atomicity assumptions and storage failure considerations.

Open primary source

S17: os.replace documentation

Python Software Foundation. Replacement semantics and platform/filesystem conditions.

Open primary source

S18: zipfile documentation

Python Software Foundation. Archive processing and extraction boundaries.

Open primary source

S19: Fix Your Timestep!

Glenn Fiedler. Simulation/render time separation and work-budget hazards.

Open primary source

S23: XAG 103: Additional channels for audio and visual cues

Microsoft. Critical information through more than one channel.

Open primary source

S24: Deterministic Lockstep

Glenn Fiedler. Input synchronization and deterministic simulation requirements.

Open primary source

S25: XAG 107: Input

Microsoft. Input/remapping accessibility considerations.

Open primary source

S26: XAG 101: Text display

Microsoft. Readable interface text and testing considerations.

Open primary source

Original program reading IDs

These inherited assignments remain part of the full study route. S entries above focus the expanded lesson tasks; they do not replace all external study.

R17: RFC 9293: Transmission Control Protocol — IETF / RFC Editor. Transport semantics; reference reading after networking foundations. Assigned source

R19: 15-462/662 Computer Graphics — Carnegie Mellon University. Rendering, geometry and animation course structure and exercises. Assigned source

R20: 15-466/666 Computer Game Programming — Carnegie Mellon University. Runtime systems and game-production programming. Assigned source

R22: SDL3 documentation — SDL project. Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples. Assigned source

R23: Physically Based Rendering, fourth edition — Matt Pharr, Wenzel Jakob, Greg Humphreys. Rendering mathematics and implementation; use chapter titles in lesson assignments. Assigned source

R26: Programming with C++ in Unreal Engine — Epic Games. Engine comparison after C++ foundations, not a beginner substitute. Assigned source

R28: Box2D documentation — Box2D project. Collision, constraints and simulation reference. Assigned source

R29: Xbox Accessibility Guidelines — Microsoft. Accessibility design and testing; not a legal-compliance certificate. Assigned source

R45: AddressSanitizer documentation — LLVM Project. Memory-error instrumentation, limitations and supported toolchains. Assigned source

R46: CS188 Introduction to Artificial Intelligence textbook — University of California, Berkeley. Search, games, decision processes, learning and probabilistic reasoning. Assigned source

R47: Dive into Deep Learning — Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola. Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions. Assigned source